

プログラムの設計方法 第二版（日本語訳）

Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

Contents

プログラムの設計方法 第二版	3
目次	3
前付け	3
本編	3
序文	6
体系的なプログラム設計	8
DrRacket とティーチング言語	10
移行可能なスキル	11
この本とその各部	11
違い点	11
プロローグ: プログラムの仕方	11
算術と算術	12
多くの方法で計算する	13
1つのプログラム、多くの定義	13
あなたはもうプログラマだ	13
I 固定サイズのデータ (Fixed-Size Data)	13
1 算術 (Arithmetic)	14
1.1 数の算術 (The Arithmetic of Numbers)	14
1.2 文字列の算術 (The Arithmetic of Strings)	14
1.3 混ぜ合わせ (Mixing It Up)	15
1.4 画像の算術 (The Arithmetic of Images)	15
1.5 ブール値の算術 (The Arithmetic of Booleans)	15
1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)	16
1.7 述語: 自分のデータを知れ (Predicates: Know Thy Data)	16
2 関数とプログラム (Functions and Programs)	16
2.1 関数 (Functions)	16
2.2 計算 (Computing)	17
2.3 関数の合成 (Composing Functions)	17
2.4 グローバル定数 (Global Constants)	17
2.5 プログラム (Programs)	17
3 プログラムの設計方法	18
3.1 関数の設計	19
情報とデータ	19
設計プロセス	22

関数の例から設計へ	23
3.2 指の運動: 関数	23
3.3 ドメイン知識	24
3.4 関数からプログラムへ	24
3.5 テストについて	25
3.6 World プログラムの設計	25
3.7 バーチャルペットの世界	26
4.1 条件式を使ったプログラミング (Programming with Conditionals)	27
4.2 条件付きの計算 (Computing Conditionally)	29
4.3 列挙 (Enumerations)	31
4.4 区間 (Intervals)	33
4.5 項目化 (Itemizations)	35
4.6 項目化を用いた設計 (Designing with Itemizations)	37
4.7 有限状態世界 (Finite State Worlds)	38
5 構造の追加 (Adding Structure)	39
6 項目化と構造 (Itemizations and Structures)	39
7 まとめ (Summary)	39
Intermezzo 1: Beginning Student Language	39
II 任意サイズのデータ (Arbitrarily Large Data)	39
Intermezzo 2: Quote, Unquote	39
III 抽象化 (Abstraction)	39
導入: 抽象化の必要性	39
14 類似性はどこにでもある (Similarities Everywhere)	40
14.1 関数における類似性 (Similarities in Functions)	41
14.2 異なる類似性 (Different Similarities)	42
14.3 データ定義における類似性 (Similarities in Data Definitions)	46
14.4 関数は値である (Functions Are Values)	48
14.5 関数を使った計算 (Computing with Functions)	49
15 抽象化の設計 (Designing Abstractions)	52
15.1 例からの抽象化 (Abstractions from Examples)	52
15.2 署名における類似性 (Similarities in Signatures)	56
15.3 単一の制御点 (Single Point of Control)	57
15.4 テンプレートからの抽象化 (Abstractions from Templates)	58
17 無名関数 (Nameless Functions)	59
17.1 lambda から生まれる関数 (Functions from lambda)	59
17.2 lambda を使った計算 (Computing with lambda)	59
17.3 lambda を使った抽象化 (Abstracting with lambda)	59
17.4 lambda を使った仕様 (Specifying with lambda)	59
17.5 lambda を使った表現 (Representing with lambda)	59

Intermezzo 3: Scope and Abstraction	60
IV 絡み合ったデータ (Intertwined Data)	60
Intermezzo 4: The Nature of Numbers	60
V 生成的再帰 (Generative Recursion)	60
Intermezzo 5: The Cost of Computation	60
VI 蓄積子 (Accumulators)	60
エピローグ: 先へ進む (Epilogue: Moving On)	60

プログラムの設計方法 第二版

How to Design Programs, Second Edition

著者: Matthias Felleisen, Robert Bruce Findler, Matthew Flatt, Shriram Krishnamurthi

© 2014 (更新版 2026) MIT Press

この資料は著作権保護されており、Creative Commons CC BY-NC-ND ライセンスの下で提供されています。

目次

前付け

- 序文 (Preface)
 - 体系的なプログラム設計
 - DrRacket とティーチング言語
 - 移行可能なスキル
 - この本とその各部
 - 違い点
- プロローグ: プログラムの仕方 (Prologue: How to Program)
 - 算術と算術
 - 入力と出力
 - 計算のさまざまな方法
 - 1つのプログラム、多くの定義
 - もう1つの定義
 - あなたはもうプログラマだ
 - 違う!

本編

I 固定サイズのデータ (Fixed-Size Data)

- 1 算術
 - 1.1 数の算術

- ▶ 1.2 文字列の算術
- ▶ 1.3 混ぜ合わせ
- ▶ 1.4 画像の算術
- ▶ 1.5 ブール値の算術
- ▶ 1.6 ブール値と混ぜ合わせ
- ▶ 1.7 述語: 自分のデータを知れ
- 2 関数とプログラム
 - ▶ 2.1 関数
 - ▶ 2.2 計算
 - ▶ 2.3 関数の合成
 - ▶ 2.4 グローバル定数
 - ▶ 2.5 プログラム
- 3 プログラムの設計方法 (How to Design Programs)
 - ▶ 3.1 関数の設計
 - ▶ 3.2 指の運動: 関数
 - ▶ 3.3 ドメイン知識
 - ▶ 3.4 関数からプログラムへ
 - ▶ 3.5 テストについて
 - ▶ 3.6 World プログラムの設計
 - ▶ 3.7 バーチャルペットの世界
- 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)
 - ▶ 4.1 条件式を使ったプログラミング
 - ▶ 4.2 条件付き計算
 - ▶ 4.3 列挙
 - ▶ 4.4 区間
 - ▶ 4.5 項目化
 - ▶ 4.6 項目化を使った設計
 - ▶ 4.7 有限状態の世界
- 5 構造の追加 (Adding Structure)
 - ▶ 5.1 位置から posn 構造へ
 - ▶ 5.2 posn を使った計算
 - ▶ 5.3 posn を使ったプログラミング
 - ▶ 5.4 構造型定義
 - ▶ 5.5 構造を使った計算
 - ▶ 5.6 構造を使ったプログラミング
 - ▶ 5.7 データの宇宙
 - ▶ 5.8 構造を使った設計
 - ▶ 5.9 World の中の構造
 - ▶ 5.10 グラフィカルエディタ

- ▶ 5.11 さらに多くのバーチャルペット
- 6 項目化と構造 (Itemizations and Structures)
 - ▶ 6.1 再び、項目化を使った設計
 - ▶ 6.2 World を混ぜる
 - ▶ 6.3 入力エラー
 - ▶ 6.4 World の検査
 - ▶ 6.5 等価述語
- 7 まとめ

Intermezzo 1: Beginning Student Language

BSL の語彙、文法、意味、エラーなど

II 任意サイズのデータ (Arbitrarily Large Data)

- 8 リスト (Lists)
- 9 自己参照データ定義を使った設計
- 10 リストの続き
- 11 合成による設計 (Design by Composition)
- 12 プロジェクト: リスト
 - ▶ 実世界データ: 辞書、iTunes
 - ▶ 単語ゲーム、ワーム、単純テトリス、完全宇宙戦争、有限状態機械
- 13 まとめ

Intermezzo 2: Quote, Unquote

III 抽象化 (Abstraction)

- 14 どこにでも類似点がある
- 15 抽象化の設計
- 16 抽象化の使用
- 17 無名関数 (Nameless Functions / lambda)
- 18 まとめ

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ (Intertwined Data)

- 19 S 式の詩 (The Poetry of S-expressions)
- 20 反復的洗練 (Iterative Refinement)
- 21 インタプリタの洗練
- 22 プロジェクト: XML の商業
- 23 同時処理
- 24 まとめ

Intermezzo 4: The Nature of Numbers

V 生成的再帰 (Generative Recursion)

- 25 非標準の再帰
- 26 アルゴリズムの設計
- 27 バリエーション
- 28 数学的な例
- 29 バックトラックするアルゴリズム
- 30 まとめ

Intermezzo 5: The Cost of Computation

VI 蓄積子 (Accumulators)

- 31 知識の喪失
- 32 蓄積子スタイル関数の設計
- 33 蓄積のさらなる使用
- 34 まとめ

エピローグ: 先へ進む (Epilogue: Moving On)

注意: 本翻訳は学習・個人利用のためのものです。サンプルコードは原文のまま保持しています。

原典: <https://htdp.org/2026-5-28/Book/index.html>

序文

Preface

序文

プロローグ：プログラムの仕方

I 固定サイズのデータ

Intermezzo 1: Beginning Student Language

II 任意サイズのデータ

Intermezzo 2: Quote, Unquote

III 抽象化

Intermezzo 3: Scope and Abstraction

IV 絡み合ったデータ

Intermezzo 4: The Nature of Numbers

V 生成的再帰

Intermezzo 5: The Cost of Computation

VI 蓄積子

エピローグ：先へ進む

多くの職業で何らかのプログラミングが必要です。会計士はスプレッドシートをプログラムし、ミュージシャンはシンセサイザーをプログラムし、著者はワードプロセッサをプログラムし、Web デザイナーはスタイルシートをプログラムします。私たちが初版のこれらの言葉を書いたとき（1995-2000）、読者はそれらを未来的だと考えたかもしれません。今や、プログラミングは必須のスキルとなり、雇用見通しを高めるという目的で、数多くの書籍、オンラインコース、K-12 カリキュラムがこのニーズに応えています。

典型的なプログラミングのコースでは、「動くまでいじくり回す」アプローチを教えます。それが動くと、学生は「動いた！」と叫んで次に進みます。悲しいことに、このフレーズはコンピューティングにおける最短の嘘でもあり、多くの人々の人生から何時間も奪ってきました。対照的に、本書は良いプログラミングの習慣に焦点を当て、プロフェッショナルおよび職業プログラマの両方を対象としています。

「良いプログラミング」とは、ソフトウェア作成へのアプローチで、最初から、すべての段階で、すべてのステップで体系的な思考、計画、理解に依拠するものを意味します。この点を強調するために、私たちは「体系的なプログラム設計」と「体系的に設計されたプログラム」について語ります。後者は、望まれる機能の根拠を明確に述べます。良いプログラミングは達成感の美的感覚も満たします。良いプログラムのエレガンスは、時の試練に耐えた詩や過去の時代の白黒写真に匹敵します。要するに、プログラミングと良いプログラミングの違いは、ダイナーのクレヨン画と美術館の油絵のようなものです。

いいえ、この本は誰をもマスターペインターには変えません。しかし、私たちがこの版を書くのに 15 年を費やしたのは、以下を信じているからに他なりません。

誰もがプログラムを設計できる

そして

誰もが創造的設計から得られる満足感を経験できる

実際、私たちはさらに進んで主張します。

プログラム設計— しかしプログラミングではない— は、リベラルアーツ教育において数学や言語スキルと同じ役割を果たすべきである。

設計を学ぶ学生が二度とプログラムに触れなくても、普遍的に有用な問題解決スキルを身につけ、深く創造的な活動を経験し、新しい形式の美を理解することを学ぶでしょう。この序文の残りでは、「体系的設計」とは何を意味するのか、誰がどのように恩恵を受けるのか、そしてそれをどのように教えるのかを詳しく説明します。

体系的なプログラム設計

プログラムは人々（ユーザーと呼ばれる）と他のプログラム（サーバーとクライアントコンポーネントの場合）と相互作用します。したがって、合理的に完全なプログラムは多くの構成要素から成ります。一部は入力扱い、一部は出力を作成し、一部はそれらの間のギャップを橋渡しします。私たちは関数を基本的な構成要素として選ぶ理由は、誰もが前代数で関数に遭遇するからであり、最も単純なプログラムがまさにそのような関数だからです。鍵は、どの関数が必要か、それらをどのように接続するか、そして基本的な材料からそれらをどのように構築するかを発見することです。

この文脈で、「体系的なプログラム設計」とは、2つの概念の混合を指します。設計レシピと反復的洗練です。私たちはMichael JacksonのCOBOL プログラム作成法、Daniel Friedmanとの再帰に関する会話、Robert Harperとの型理論、Daniel Jacksonとのソフトウェア設計から着想を得ました。設計レシピは著者らの創作であり、ここでは後者の使用を可能にします。

問題分析からデータ定義へ 表現しなければならない情報と、それが選択したプログラミング言語でどのように表現されるかを特定せよ。データ定義を定式化し、例で説明せよ。 署名、目的文、ヘッダ 望まれる関数がどのような種類のデータを消費し、生成するかを述べよ。その関数何何を計算するのかという問いに対する簡潔な答えを定式化せよ。署名に沿ったスタブを定義せよ。 関数の例 関数の目的を説明する例に取り組み。 関数のテンプレート データ定義を関数のアウトラインに翻訳せ

よ。関数定義 関数のテンプレートの隙間を埋めよ。目的文と例を活用せよ。テスト 例をテストとして明確に述べ、関数がすべてに合格することを確認せよ。これにより誤りが発見される。テストはまた、必要が生じたときに他の人が定義を読み理解するのを助ける—そしてそれは深刻なプログラムでは必ず生じる。

図 1: 関数設計レシピの基本ステップ

設計レシピは完全なプログラムと個々の関数の両方に適用されます。本書は完全なプログラムのための2つのレシピだけを扱います。GUIを持つプログラム用とバッチプログラム用です。一方、関数用の設計レシピにはさまざまなバリエーションがあります。数値のような原子的なデータ形式用、さまざまな種類のデータの列挙用、固定された方法で他のデータを複合するデータ用、有限だが任意に大きいデータ用、など。

関数レベルの設計レシピは共通の設計プロセスを共有します。図 1 はその6つの必須ステップを示しています。各ステップのタイトルは期待される成果を指定し、「コマンド」は主要な活動を示唆します。例はほぼすべての段階で中心的な役割を果たします。

各ステップには鋭い質問が付随しており、本書の6つのパートを通じて導入されます。特定のステップ—例えば関数の例の作成やテンプレート—では、質問はデータ定義に訴えかけるかもしれません。答えはほぼ自動的に中間成果物を作成します。

このアプローチの新しさは、初心者レベルのプログラムのための中間成果物の作成にあります。初心者が詰まったとき、専門家や講師は既存の中間成果物を検査できます。検査は設計プロセスの一般的な質問を使う可能性が高く、したがって初心者に自分で修正させることになります。そしてこの自己強化プロセスこそが、プログラミングとプログラム設計の重要な違いです。

反復的洗練は、問題が複雑で多面的であるという問題に対処します。一度にすべてを正しくすることはほぼ不可能です。代わりに、コンピュータ科学者はこの設計問題に取り組むために物理科学から反復的洗練を借用します。本質的に、反復的洗練は、最初にすべての非本質的な詳細を取り除き、残された中核問題の解決策を見つけることを推奨します。洗練ステップは、これらの省略された詳細の1つを追加し、既存の解決策を可能な限り使用して拡張された問題を再解決します。これらの洗練ステップの繰り返し（反復とも呼ばれる）は、最終的に完全な解決策につながります。

この意味で、プログラマはミニ科学者です。科学者は世界の理想化されたバージョンのための近似モデルを作成し、それについての予測を行います。モデルの予測が当たっている限り、すべてはうまくいきます。予測された出来事が実際のもthingと異なる場合、科学者はその不一致を減らすためにモデルを改訂します。同様に、プログラマにタスクが与えられたとき、彼らは最初の設計を作成し、それをコードにし、実際のユーザーで評価し、プログラムの振る舞いが望まれる製品に密接に一致するまで設計を反復的に洗練します。

本書は2つの異なる方法で反復的洗練を導入します。洗練による設計がプログラムの設計が複雑になると有用になるため、本書は問題がある程度の難易度を獲得した第4部でこ

の技法を明示的に導入します。さらに、私たちは第1部から第3部を通じて同じ問題のますます複雑なバリエーションを述べるために反復的洗練を使用します。つまり、中核問題を選び、1つの章でそれを扱い、その後、後続の章で新たに導入された概念に合致する詳細で類似の問題を提示します。

DrRacket とティーチング言語

プログラムを設計することを学ぶには、繰り返しのハンズオン練習が必要です。ピアノを弾かずにピアノ奏者になれないのと同じように、実際のプログラムを作成して正しく動作させることなしにプログラム設計者にはなれません。したがって、本書には最小限のソフトウェアサポートが付属しています。プログラムを書き留めるための言語と、プログラムをワード文書のように編集し、プログラムを実行できるプログラム開発環境です。

多くの人が「コードの書き方を学びたい」と言い、次にどのプログラミング言語を学ぶべきかを尋ねます。いくつかのプログラミング言語が得ている注目を考えると、この質問は驚くべきことではありません。しかし、それは全く不適切でもあります。初心者を対象としないコースでは、既製の言語を設計レシピとともに使用できるかもしれません。現在流行のプログラミング言語でプログラミングを学ぶことは、学生を最終的な失敗に導くことがよくあります。この世界での流行は非常に短命です。「Xで素早くプログラミング」本やコースは、次の流行の言語に移行できる原則を教えることに失敗することが多いです。さらに悪いことに、言語自体が、解決策を表現するレベルでもプログラミングミスに対処するレベルでも、移行可能なスキルの習得から気を逸らします。

対照的に、プログラムを設計することを学ぶことは、主に原則の研究と移行可能なスキルの習得についてです。理想的なプログラミング言語はこれら2つの目標をサポートしなければなりませんが、既製の産業用言語はそうしません。重要な問題は、初心者が言語の多くを知る前に間違いを犯すことです。しかしプログラミング言語は、プログラマがすでに言語全体を知っているかのように常にこれらのエラーを診断します。その結果、診断レポートは初心者をしばしば困惑させます。

私たちの解決策は、私たち自身の仕立てられたティーチング言語から始めることです。「Beginning Student Language」またはBSLと呼ばれます。この言語は本質的に、学生が前代数のコースで習得する「外国語」です。関数定義、関数適用、条件式のための記法を含みます。また、式を入れ子にすることもできます。この言語は非常に小さいので、言語全体の観点からのエラー診断でも、前代数しか持たない読者にとってまだアクセス可能です。

構造的設計原則を習得した学生は、次に「Intermediate Student Language」と他の高度な方言（総称して*SL）に進むことができます。本書はこれらの方言を使用して抽象化と一般再帰の設計原則を教えます。私たちは、このような一連のティーチング言語を使用することが、幅広い専門プログラミング言語（JavaScript、Python、Ruby、Javaなど）でプログラムを作成するための優れた準備を読者に提供すると固く信じています。

注：ティーチング言語は Racket で実装されています。Racket はプログラミング言語を構築するためのプログラミング言語として私たちが構築したものです。Racket はラボから現実世界に逃れ出ました。それはプログラミング言語です。

(続き：DrRacket の使用、ティーチング言語の選択など詳細)

移行可能なスキル

(中略：設計レシピから得られる普遍的な問題解決スキル、創造的活動としてのプログラミングの価値について)

この本とその各部

本書は 6 つの主要なパートと 5 つの Intermezzo に分かれています。各パートは設計レシピの特定の側面に焦点を当て、徐々に複雑さを増していきます。

- Part I: 固定サイズのデータ – 基本的な算術から設計レシピの導入まで。
- Part II: 任意サイズのデータ – リストと自己参照データ定義。
- Part III: 抽象化 – 類似性の発見と高階関数。
- Part IV: 絡み合ったデータ – 複雑なデータ定義の相互作用。
- Part V: 生成的再帰 – 構造的再帰を超えたアルゴリズム。
- Part VI: 蓄積子 – 効率的な再帰のための追加の知識の保持。

各 Intermezzo では、言語の形式的な側面（文法、意味、計算モデル）を扱います。

違い点

(初版との違いの説明：より明確な設計レシピ、プロジェクトの強化など)

謝辞 (Acknowledgments)

… (詳細は原典を参照。多くの貢献者への感謝)

本翻訳は原典の内容を忠実に日本語化したものです。サンプルコードは原文を維持しています。

原典 URL: https://htdp.org/2026-5-28/Book/part_preface.html

プロローグ：プログラムの仕方

Prologue: How to Program

(目次リンクは省略)

あなたが小さな子供だったとき、親はあなたに数を数え、指で簡単な計算をすることを教えました。「 $1 + 1$ は 2 」;「 $1 + 2$ は 3 」; など。そして「 $3 + 2$ は？」と尋ね、あなたは片手の指を数えました。彼らはプログラムし、あなたは計算しました。そしてある意味で、それが本当にプログラミングとコンピューティングのすべてです。

今度は役割を交代する時です。DrRacket を起動しましょう。そうすると DrRacket のウィンドウが表示されます。「Language」メニューから「Choose language」を選び、「How to Design Programs」の「Teaching Languages」から「Beginning Student」(BSL) を選択し、OK をクリックして DrRacket を設定します。

これで準備完了です。あなたがプログラムし、DrRacket ソフトウェアが子供の役割を果たします。

最も単純な計算から始めましょう。DrRacket の上部に次のように入力します：

(+ 1 1)

RUN をクリックすると、下部に 2 が表示されます。

(図：DrRacket の画面)

それほど単純なのがプログラミングです。DrRacket を子供のように質問し、DrRacket があなたのために計算します。一度に複数のリクエストを処理させることもできます：

(+ 2 2)

(* 3 3)

(- 4 2)

(/ 6 2)

RUN をクリックすると、期待される結果 4 9 2 3 が表示されます。

少しペースを落として用語を導入しましょう。

DrRacket の上半分は「定義領域 (definitions area)」と呼ばれます。この領域でプログラムを作成します。これを編集と呼びます。定義領域に単語を追加したり変更したりすると、左上に SAVE ボタンが表示されます。初めて SAVE をクリックすると、DrRacket はファイルを保存するための名前を尋ねます。一度ファイルに関連付けられると、SAVE をクリックすると定義領域の内容が安全にファイルに保存されます。

プログラムは式 (expressions) で構成されます。数学で式に出会ったことがあるでしょう。今のところ、式は単なる数値か、左括弧「(」で始まり対応する右括弧「)」で終わるものです—DrRacket はその括弧の間の領域を強調表示します。

RUN をクリックすると、DrRacket は定義領域の式を評価し、その結果を対話領域 (interactions area) に表示します。その後、DrRacket はプロンプト (>) であなたのコマンドを待ちます。プロンプトが表示されたら、追加の式を入力でき、DrRacket は定義領域と同じようにそれを評価します。

このようなプログラミングとコンピューティングを DrRacket で簡単に探求できます。

算術と算術

プログラミングが数と算術だけだったら、数学と同じくらい退屈でしょう。幸い、プログラミングには数値以外にも多くのものがあります。テキスト、真偽値、画像などです。

BSL で知っておくべき最初のことは、テキストは二重引用符（ “ ）で囲まれた任意のキーボード文字のシーケンスであるということです。これを文字列（string）と呼びます。したがって、

```
"hello world"
```

は完全に正しい文字列であり、DrRacket がこの文字列を評価すると、対話領域で数値と同じようにそのままエコーバックします。

BSL には数の算術に加えて、文字列の算術もあります。以下に 2 つの例を示します：

```
(string-append "hello" "world")  
; "helloworld"
```

```
(string-append "hello " "world")  
; "hello world"
```

+ と同様に、string-append は演算子です。2 番目の文字列を最初の文字列の末尾に追加して新しい文字列を作ります。最初の例が示すように、2 つの文字列の間に何も挿入せずに文字通り結合します。空白、カンマ、何もありません。

(以下、画像の算術、ブール値、条件式、関数の定義などプロローグの続きが展開されます。)

多くの方法で計算する

(条件式の導入例)

1 つのプログラム、多くの定義

(define を使った定数と関数の導入)

あなたはもうプログラマだ

ここまで来たら、あなたはすでに小さなプログラムを書けるプログラマです。以降の章で設計レシピを学び、体系的にプログラムを構築していきます。

注意：サンプルコードは原文のままです。

原典：https://htdp.org/2026-5-28/Book/part_prologue.html

I 固定サイズのデータ (Fixed-Size Data)

本書の第 I 部では、基本的な原子データ（数値、文字列、画像、真偽値）を操作する「算術」と、それらを組み合わせた最も単純なプログラムである「関数」について学び、体系的なプログラム設計の最初の一步を踏み出します。

1 算術 (Arithmetic)

BSL (Beginning Student Language) における算術は、前置表記 (Prefix Notation) で記述されます。基本規則： 1. 開き括弧 (を書く 2. 基本演算の名前 (+ など) を書く 3. 引数をスペースで区切って書く 4. 閉じ括弧) を書く

例：

```
(+ 1 2)
```

ネスト (入れ子) された例：

```
(+ 1 (+ 1 (+ 1 1) 2) 3 4 5)
```

評価は、まず引数となる式をすべて評価し、その結果のデータ片を演算に与えて計算します。

```
(+ 1 2)
```

```
==
```

```
3
```

1.1 数の算術 (The Arithmetic of Numbers)

BSL は自然数、整数、有理数、実数、複素数をサポートし、正確数 (Exact) と不正確数 (Inexact) を区別します。

演算例： +, -, *, /, abs, add1, ceiling, denominator, exact->inexact, expt, floor, gcd, log, max, numerator, quotient, random, remainder, sqr, tan

```
> (sin 0)
```

```
0
```

また、円周率 pi やオイラーの定数 e も事前定義されています。不正確数 (近似値) には #i プレフィックスが付きます (例：#i1.4142135623730951)。

Exercise 1. 定義領域に以下を追加し、デカルト平面上の点 (x, y) から原点 (0, 0) までの距離を計算する式を記述せよ。

```
(define x 3)
```

```
(define y 4)
```

1.2 文字列の算術 (The Arithmetic of Strings)

文字列はダブルクォーテーション " で囲まれた文字列です。

演算例： string-append, string-length, substring, string-ith

```
(string-append "hello" " " "world")
```

```
; "hello world"
```

Exercise 2. prefix と suffix という名前の文字列定数を定義し、それらを結合する式を記述せよ。

1.3 混ぜ合わせ (Mixing It Up)

文字列と数値を相互に変換したり、型を操作する演算です。

演算例: number->string, string->number, string-length

```
(string-length "42")  
==  
2
```

Exercise 3. 文字列 `str` と数値 `i` を定義し、`str` の `i` 番目の位置に `_` を挿入する式を記述せよ。

```
(define str "helloworld")  
(define i 5)
```

Exercise 4. 文字列 `str` と数値 `i` が与えられたとき、`i` 番目の文字を削除する式を記述せよ。

1.4 画像の算術 (The Arithmetic of Images)

2http/image ティーチパックを使用すると、画像をデータとして操作できます。

演算例: circle, rectangle, empty-scene, place-image, overlay, image-width, image-height

Exercise 5. 1 枚の画像のピクセル数を計算する式を作成せよ。

```
(define cat <画像>)
```

Exercise 6. 任意の画像 `cat` が与えられたとき、それが縦長か、横長か、正方形かを判定する式を作成せよ。

Exercise 7. 以下の画像作成を試せ。

```
(circle 10 "solid" "red")  
(rectangle 10 20 "outline" "blue")
```

Exercise 8. 青いバナーの上に赤い文字でテキストを表示する画像を生成する式を記述せよ。

1.5 ブール値の算術 (The Arithmetic of Booleans)

真偽値 (`#true`, `#false`) と、論理演算 (`and`, `or`, `not`)、および比較述語 (`<`, `>`, `=`, `string=?`) です。

```
(or #false #true)  
==  
#true
```

Exercise 9. 画像 `in` に対して、それが縦長であるか、または横長であるかを判定するブール値の式を作成せよ。

1.6 ブール値と混ぜ合わせ (Mixing It Up with Booleans)

条件に応じて処理を分岐させるための `if` 式です。

```
(if (string=? "red" "green") "stop" "go")  
==  
"go"
```

Exercise 10. 与えられた画像が縦長なら `"tall"`、横長なら `"wide"`、正方形なら `"square"` を返す `if` 式を記述せよ。

1.7 述語: 自分のデータを知れ (Predicates: Know Thy Data)

データ型を検査するための述語です。 `number?`, `string?`, `image?`, `boolean?`, `any?`

Exercise 11. `number?` や `string?` の動作を確認する式を記述せよ。

Exercise 12. 値 `v` が与えられたとき、それが画像ならその面積、文字列ならその長さ、数値ならその値自体を返す条件分岐式を記述せよ。

2 関数とプログラム (Functions and Programs)

プログラムの半分は算術であり、もう半分はそれらを再利用可能にする「関数」です。

2.1 関数 (Functions)

`define` を使って新しい関数を定義します。

```
(define (f x) (+ x 10))
```

Exercise 13. 与えられた文字列の最初の文字を返す関数 `string-first` を定義せよ。

Exercise 14. 与えられた文字列の最後の文字を返す関数 `string-last` を定義せよ。

Exercise 15. 2つのブール値 `sunny` と `friday` を消費し、金曜日で晴れの日であるかを判定する関数 `play` を定義せよ。

Exercise 16. 画像を消費し、そのピクセル面積を計算する関数 `image-area` を定義せよ。

Exercise 17. 画像を消費し、それが縦長であるかを判定する関数 `image-classify` を定義せよ。

Exercise 18. 2つの文字列を消費し、それらを `"_"` で結合する関数 `string-join` を定義せよ。

Exercise 19. 文字列 `str` と位置 `i` を消費し、`i` 番目に `"_"` を挿入する関数 `string-insert` を定義せよ。

Exercise 20. 文字列 `str` と位置 `i` を消費し、`i` 番目の文字を削除する関数 `string-delete` を定義せよ。

2.2 計算 (Computing)

関数の評価は、実引数 (Argument) を仮引数 (Parameter) に置き換える「代入モデル」に従って行われます。

```
(f 2)
==
(+ 2 10)
==
12
```

Exercise 21. (ff 2) がどのように評価されるか代入ステップを記述せよ。

```
(define (ff x) (* 10 x))
```

Exercise 22. (distance 3 4) の評価ステップを記述せよ。

Exercise 23. (string-first "hello") の評価ステップを記述せよ。

Exercise 24. (implies #true #false) の評価ステップを記述せよ。

Exercise 25. 画像分類関数の評価ステップを記述せよ。

Exercise 26. (string-insert "hello" 2) の評価ステップを記述せよ。

2.3 関数の合成 (Composing Functions)

複雑なタスクは、単純な関数を組み合わせることで解決します。

```
(define (profit price)
  (- (revenue price) (cost price)))
```

Exercise 27. 映画館のチケット価格と利益の関係をモデル化するプログラムを構築せよ。

Exercise 28. チケット価格ごとの利益をシミュレーションし、最適なチケット価格を見つけよ。

Exercise 29. コスト構造が変化した場合の最適なチケット価格を再計算せよ。

2.4 グローバル定数 (Global Constants)

変更に強いプログラムを作るために、マジックナンバーをグローバル定数として定義します。

```
(define BASE-PRICE 5)
```

Exercise 30. 映画館の利益計算プログラムにおける定数（基本価格、固定費、変動費など）を定数定義として分離せよ。

2.5 プログラム (Programs)

BSL では 2http/batch-io を使ったバッチ処理プログラムや、2http/universe を使った対話型（イベント駆動）プログラム (big-bang) を設計できます。

```
(big-bang 0  
  [on-tick subl]  
  [to-draw render]  
  [on-key handle-key])
```

Exercise 31. ファイルからテキストを読み込んで処理するバッチプログラムを作成せよ。

Exercise 32. big-bang を使って、画面を動く UF0 やロケットのアニメーションプログラムを作成せよ。

3 プログラムの設計方法

本書の最初の数章では、プログラミングを学ぶには多くの概念をある程度習得する必要があることを示してきました。一方では、プログラミングには言語が必要です。つまり、計算したいことを伝えるための表記法です。プログラムを定式化するための言語は人工的な構成物ですが、プログラミング言語の習得は自然言語の習得といくつかの要素を共有しています。どちらも語彙、文法、そして「句」が何を意味するかの理解を伴います。

他方では、問題文からプログラムに至る方法を学ぶことが重要です。問題文の中で何が関連していて、何を無視できるかを判断する必要があります。プログラムが何を消費し、何を生成し、入力と出力をどのように関連付けるかを明らかにする必要があります。選択した言語とそのティーチパックが、プログラムが処理するデータに対する特定の基本演算を提供しているかどうかを知る（または調べる）必要があります。提供していない場合は、それらの演算を実装する補助関数を開発しなければならないかもしれません。最後に、プログラムができたなら、それが実際に意図した計算を行っているかどうかを確認しなければなりません。そして、これはあらゆる種類のエラーを明らかにする可能性があり、それらを理解して修正できなければなりません。

これらすべてはかなり複雑に聞こえ、なぜただ実験しながら適当にやって、結果がまあまあならそのままにしておけばいいのかと思うかもしれません。このプログラミングへのアプローチはしばしば「ガレージプログラミング」と呼ばれ、多くの場合に成功します。時にはスタートアップ企業の立ち上げの足がかりになることもあります。しかし、スタートアップはその「ガレージ努力」の結果を販売することはできません。なぜなら、元のプログラマとその友人しか使えないからです。

良いプログラムには、それが何をするか、どのような入力を期待するか、何を生成するかを説明する短い書き込みが付属しています。理想的には、それが実際に動作することをある程度保証するものも付きます。最良の場合、プログラムの問題文へのつながりが明白なので、問題文への小さな変更をプログラムへの小さな変更簡単に翻訳できます。ソフトウェアエンジニアはこれを「プログラミング製品」と呼びます。

このすべての追加作業が必要なのは、プログラマが自分自身のためにプログラムを作成するわけではないからです。プログラマは、他のプログラマが読むためにプログラムを書きます。そして時折、人々は仕事をするためにこれらのプログラムを実行します。

「他の」という言葉には、通常、若い頃の自分がプログラムの制作に込めた思考のすべてを思い出すことができない年上のバージョンのプログラマも含まれます。

ほとんどのプログラムは、協力し合う関数の大規模で複雑な集合であり、誰もが1日でこれらすべての関数を書くことはできません。プログラマはプロジェクトに参加し、コードを書き、プロジェクトを去ります。他の人がそのプログラムを引き継いで作業します。もう一つの困難は、プログラマのクライアントが、本当に解決してほしい問題について考えを変えがちだということです。彼らは通常、ほぼ正しく理解していますが、多くの場合、いくつかの詳細を間違えます。さらに悪いことに、プログラムのような複雑な論理的構成物は、ほとんど常に人間のエラーに悩まされます。要するに、プログラマは間違いを犯します。最終的に誰かがこれらのエラーを発見し、プログラマはそれらを修正しなければなりません。彼らは1ヶ月前、1年前、または20年前のプログラムを読み直して変更する必要があります。

Exercise 33. 「year 2000」問題を調べよ。

ここでは、ステップバイステップのプロセスと、問題データを中心にプログラムを組織する方法を統合した設計レシピを提示します。長い間空白の画面を見つめ続けるのが嫌いな読者にとって、この設計レシピは体系的な方法で進捗を達成する方法を提供します。他の人をプログラム設計に教える人にとって、レシピは初心者の困難を診断するための道具です。他の人々にとっては、このレシピは医学、ジャーナリズム、工学など他の分野にも適用できるものかもしれません。本物のプログラマになりたい人にとって、設計レシピは既存のプログラムを理解し作業するための方法も提供します—ただし、すべてのプログラマがこのような設計レシピのような方法を使ってプログラムを考え出すわけではありません。この章の残りは、設計レシピの世界への最初の一步を捧げます。以降の章とパートは、レシピを何らかの形で洗練・拡張していきます。

3.1 関数の設計

情報とデータ

プログラムの目的は、ある情報を消費し、新しい情報を生成する計算プロセスを記述することです。この意味で、プログラムは数学の教師が生徒に与える指示のようなものです。しかし生徒とは異なり、プログラムは数値以上のものを扱います。ナビゲーション情報を計算し、人の住所を調べ、スイッチを入れ、ビデオゲームの状態を検査します。これらすべての情報は現実世界の一部—しばしばプログラムのドメインと呼ばれる—から来ており、プログラムの計算結果はこのドメイン内のさらなる情報を表します。

情報は私たちの説明で中心的な役割を果たします。情報をプログラムのドメインに関する事実として考えてください。家具カタログを扱うプログラムにとって、「5本脚のテーブル

ル」や「2メートル四方の正方形テーブル」は情報の断片です。ゲームプログラムは異なる種類のドメインを扱い、「5」はあるオブジェクトがキャンバスのある部分から別の部分へ移動する際の、クロックティックあたりのピクセル数を指すかもしれません。また、給与計算プログラムは「5つの控除」を扱う可能性が高いでしょう。

プログラムが情報を処理するためには、それをプログラミング言語の何らかの形のデータに変換しなければなりません。そしてデータを処理し、完了したら結果のデータを再び情報に変換します。対話型プログラムはこれらのステップをさらに混ぜ合わせ、必要に応じて世界からより多くの情報を取得し、その間に情報を配信するかもしれません。

私たちはBSLとDrRacketを使用するので、情報からデータへの変換について心配する必要はありません。DrRacketのBSLでは、関数をデータに直接適用して何を生成するかを観察できます。その結果、情報をデータに変換したりその逆を行う関数を書くという深刻なニワトリと卵の問題を避けられます。単純な種類の情報の場合、そのようなプログラム部品の設計は些細です。単純な情報以外のものについては、例えばパーズングを知る必要があります、それは直ちにプログラム設計における多くの専門知識を必要とします。

ソフトウェアエンジニアは、BSLとDrRacketがデータ処理を情報→データのパーズングやデータ→情報への変換から分離する方法をmodel-view-controller (MVC) というスローガンで表現します。実際、十分に設計されたソフトウェアシステムはこの分離を強制することが今や受け入れられた知恵となっています。入門書の多くは依然としてそれらを混同していますが、したがって、BSLとDrRacketで作業することで、プログラムの核心の設計に集中でき、そこに十分な経験を積んだ後、情報-データ変換部分の設計を学ぶことができます。

ここでは、データと情報の分離を示すために、2つのプリインストール済みティーチパックを使用します：2htdp/batch-ioと2htdp/universeです。この章から、バッチプログラムと対話型プログラムのための設計レシピを開発し、完全なプログラムがどのように設計されるかのアイデアを提供します。完全なプログラミング言語のティーチパックは完全なプログラムのためのより多くの文脈を提供しており、設計レシピを適切に適応させる必要があることを心に留めておいてください。

図 15: 情報からデータへ、そして戻る

情報とデータの中心的な役割を考えると、プログラム設計はそれらの間のつながりから始めなければなりません。具体的には、私たちプログラマは、選択したプログラミング言語を使って関連する情報の断片をデータとしてどのように表現し、データをどのように解釈して情報とするかを決定しなければなりません。図 15はこの考えを抽象的な図で説明しています。

この考えを具体的にするために、いくつかの例を見てみましょう。数値の形で情報を消費・生成するプログラムを設計しているとします。表現の選択は簡単ですが、解釈には42のような数値がドメインで何を表すかを説明する必要があります：

- 42 は画像のドメインでは上端からのピクセル数を表すかもしれない
- 42 はシミュレーションやゲームオブジェクトが移動するクロックティックあたりのピクセル数を表すかもしれない
- 42 は物理学のドメインでは華氏、摂氏、または絶対温度目盛りでの温度を意味するかもしれない
- 42 はプログラムのドメインが家具カタログの場合、あるテーブルのサイズを指定するかもしれない
- 42 は単に文字列内の文字数を数えるかもしれない

鍵は、数値を情報として数値をデータとして（そしてその逆へ）どのように移行するかを知ることです。

この知識はプログラムを読むすべての者にとって非常に重要なので、私たちはしばしばそれをコメントの形で書き留め、データ定義と呼びます。データ定義には2つの目的があります。第一に、意味のある単語を使ってデータのあるコレクション（クラス）に名前を付けます。第二に、そのクラスの要素をどのように作成し、任意のデータ片がそのクラスに属するかどうかをどのように判断するかを読者に伝えます。

例：

```
; Temperature は Number
; 摂氏での温度を表す
```

このデータ定義は、任意の数値を温度として解釈できることを述べています。-400 も Temperature です。読者はデータ定義から、-400 が有効な Temperature であることを理解します。

あるデータ定義が与えられたとき、読者は「このクラスに属するデータ片の例を挙げよ」と尋ねられるべきです。Temperature の場合、-273、0、100 は良い例です。読者はまた「この特定のデータはどのクラスに属するか？」と尋ねられるべきです。42 は Number、String、Image、Boolean のいずれでもありません。なぜなら文字列は Temperature に属さないからです。そして、Temperature のサンプルを作れと言われたら、-400 のようなものを思いつくかもしれません。

可能な最低温度が約-273℃であることを知っている場合、この知識をデータ定義で表現できるかどうかを疑問に思うかもしれません。データ定義は実際にはクラスの英語による記述に過ぎないので、確かにここに示したよりもはるかに正確な方法で温度のクラスを定義できます。本書では、このようなデータ定義に様式化された英語を使用し、次の章で「-274 より大きい」などの制約を課すためのスタイルを導入します。

これまで、4つのデータクラスの名前に出会いました：Number、String、Image、Boolean です。この知識があれば、新しいデータ定義を定式化することは、既存のデータ形式に新しい名前を導入することに他なりません。例えば「temperature」を数値に与えることです。この限られた知識でも、私たちの設計プロセスの概要を説明するのに十分です。

設計プロセス

入力情報をデータとして表現する方法と、出力データを情報として解釈する方法を理解したら、個々の関数の設計は簡単なプロセスに従って進められます。

1. 情報をデータとしてどのように表現したいかを表現せよ。1行のコメントで十分です：

； センチメートルを表すために数値を使います。

2. プログラムの成功する設計に関連すると考えるデータクラスに対して、Temperatureのようなデータ定義を定式化せよ。
3. 署名、目的文、関数ヘッダを書け。

関数署名 は、設計の読者にあなたの関数がいくつの入力を消費し、それらがどのクラスから来ているか、何種類のデータを生成するかを伝えるコメントです。以下は3つの例です：

```
； String -> String
； String -> Number
； Number String Image -> Image
```

目的文 は、関数が何をするかを1文で述べるコメントです。良い目的文は、関数を読まずに何をするかを誰でも理解できるようにします。

ヘッダ（スタブとも呼ばれる）は単純化された関数定義です。署名内の各入力クラスに対して1つの変数名を選び、関数の本体は出力クラスからの任意のデータ片にできます。

```
(define (f a-string) 0)
(define (g n) "a")
(define (h num str img) (empty-scene 100 100))
```

パラメータ名は、パラメータが表すデータの種別を反映すべきです。時には、パラメータの目的を示唆する名前を使いたいと思うかもしれません。

目的文を定式化するとき、パラメータ名を使って何が計算されるかを明確にすると役立つことがよくあります。

4. 関数の例を作成せよ。

各関数について、少なくとも2～3個の具体例を挙げ、入力と期待される出力を書きましょう。これを「指の運動」と呼びます。

```
； given: "hello", expect: "h"
； given: "world", expect: "w"
```

5. テンプレートを書け。

データ定義から関数のアウトラインを導き出します。条件分岐が必要な場合は `cond` の骨組みを、構造体が必要なら選択子を使った骨組みを書きます。

6. 関数定義を完成させよ。

テンプレートを埋め、目的文と例を使って実際の計算ロジックを記述します。

7. テストを行え。

例を `check-expect` に変換し、すべてが通ることを確認します。

図 1: 関数設計レシピの基本ステップ（前述の 6 ステップの要約図）

このプロセスは初心者が進捗を確実にし、エラーを早期に発見するのを助けます。

関数の例から設計へ

ここでは、文字列の最初の文字を抽出する簡単な関数 `string-first` を設計する例を示します。

（コード例は原文のまま）

```
; String -> String
; 文字列 s の最初の文字を抽出する
(define (string-first s) "a")

; 例:
; given: "hello", expect: "h"
; given: "world", expect: "w"
; given: " ", expect: " "

(check-expect (string-first "hello") "h")
(check-expect (string-first "world") "w")
```

（以降の章の例も同様に設計レシピを適用して進めます。）

3.2 指の運動：関数

この節の最初のいくつかの練習問題は、前の「関数」節のものとほぼコピーです。ただし、後者で「define」という単語を使っているところを、ここでは「design」という単語を使っています。この違いが意味するのは、設計レシピに従ってこれらの関数を作成し、解答には関連するすべての部品を含めるべきだということです。

節のタイトルが示唆するように、これらはプロセスを身体化するための練習問題です。ステップが第二の天性になるまで、1つも飛ばさないでください。飛ばすと簡単に避けられるエラーにつながります。プログラミングには複雑なエラーの余地がたくさん残されています。ばかばかしいエラーに時間を無駄にする必要はありません。

Exercise 34. 関数 `string-first` を設計せよ。文字列 `s` から最初の文字を抽出する。

Exercise 35. 関数 `string-last` を設計せよ。文字列 `s` から最後の文字を抽出する。

Exercise 36. 関数 `image-area` を設計せよ。画像 `img` の面積を計算する。

Exercise 37. 関数 `string-rest` を設計せよ。文字列 `s` から最初の文字を除いた残りを返す。

Exercise 38. 関数 `string-remove-last` を設計せよ。文字列 `s` から最後の文字を除いた残りを返す。

(本書ではさらに多くの練習問題が続き、すべて設計レシピの適用を練習するものです。)

3.3 ドメイン知識

関数の本体をコード化するのにどのような知識が必要かを不思議に思うのは自然です。少し反省すると、このステップにはプログラムのドメインに対する適切な把握が必要であることがわかります。実際、そのようなドメイン知識には2つの形態があります：

1. 外部ドメインからの知識—数学、音楽、生物学、土木工学、芸術など。プログラマはコンピューティングのすべての応用ドメインを知ることはできないため、ドメイン専門家と問題を議論できるように、さまざまな応用分野の言語を理解する準備ができていなければなりません。数学は多くの（すべてではない）ドメインの交差点にあります。したがって、プログラマはドメイン専門家と問題を処理する際に新しい言語を習得しなければならないことがよくあります。
2. ティーチング言語とそのライブラリに関する知識。プログラマは、与えられた問題に対して言語が何を提供しているかを知る必要があります。

設計プロセスの中で、ドメイン知識は主にステップ5（関数定義を完成させる）で必要になりますが、良い例を作成するためにも役立ちます。

3.4 関数からプログラムへ

すべてのプログラムが単一の関数定義から成るわけではありません。いくつかの関数が必要とするものもあります。多くのものは定数定義も使用します。いずれにせよ、すべての関数を体系的に設計することは常に重要ですが、グローバル定数や補助関数は設計プロセスを少し変えます。

グローバル定数を定義した場合、関数は結果を計算するためにそれらを使用するかもしれませんが。それらの存在を自分に思い出させるために、テンプレートにこれらの定数を追加するとよいでしょう。結局のところ、それらは関数定義に寄与し得るものの目録に属します。

複数関数のプログラムは、対話型プログラムが自動的にキーイベントやマウスイベントを処理する関数、状態を画像として描画する関数などを必要とするために生じます。各補助関数も設計レシピに従って設計すべきです。

プログラム全体にはトップレベルの目的文も書くべきです。

3.5 テストについて

テストはすぐに労働集約的な作業になります。小さなプログラムを対話領域で実行するのは簡単ですが、多くの機械的な作業と複雑な検査が必要です。プログラマがシステムを成長させるにつれ、多くのテストを実施したいと思うようになります。やがてこの作業は圧倒的になり、プログラマはそれを怠るようになります。同時に、テストは基本的な欠陥を発見・防止するための最初の道具です。ずさんなテストはすぐにバグのある関数—隠れた問題を抱えた関数—につながり、バグのある関数はプロジェクトをしばしば複数の方法で遅らせます。

したがって、手動でテストを行うのではなく、テストを機械化することが重要です。多くのプログラミング言語と同様に、BSL にはテスト機能が含まれており、DrRacket はこの機能を認識しています。

check-expect を使って例を自動テストに変換します：

```
(check-expect (string-first "hello") "h")
```

DrRacket は RUN 時にすべてのテストを実行し、結果を表示します。失敗したテストは赤で表示され、デバッグに役立ちます。

良いテストには以下が含まれます： - 典型的なケース - 境界値（空文字列、長さ 1 の文字列など） - 特殊ケース

テストはまた、プログラムのドキュメントとしても機能します。

3.6 World プログラムの設計

前の章では 2htdp/universe ティーチパックをアドホックな方法で紹介しましたが、この節では設計レシピがどのようにして world プログラムを体系的に作成するのを助けるかを示します。まず、データ定義と関数署名に基づく 2htdp/universe ティーチパックの簡単な要約から始めます。次に、world プログラムのための設計レシピを詳述します。

ティーチパックは、プログラマが世界の状態を表すデータ定義と、すべての可能な世界状態に対して画像を作成する方法を知っている関数 render を開発することを期待します。プログラムの必要性に応じて、プログラマは次にクロックティック、キー入力、マウスイベントに応答する関数を設計しなければなりません。最後に、対話型プログラムは big-bang を使って状態、描画関数、イベントハンドラを結び付ける必要があります。

world プログラム設計レシピの概要：

1. 世界の状態を表すデータ定義を書く。
2. render 関数の署名と目的文を書く：WorldState -> Image
3. 必要に応じて on-tick、on-key、on-mouse などのハンドラの署名を書く。
4. 各関数について設計レシピを適用する。
5. big-bang でプログラムを組み立てる。

例として、シンプルなカウンターやアニメーションを設計できます。

3.7 バーチャルペットの世界

この練習問題の節は、バーチャルペットゲームの最初の2つの要素を紹介します。キャンバスを横切って歩き続ける猫の表示から始まります。もちろん、歩き続けると猫は不幸になり、その不幸が現れます。すべてのペットと同様に、撫でて少し助けるか、餌を与えてたくさん助けることができます。

まず、私たちのお気に入りの猫の画像から始めましょう：

```
(define cat1 <画像>)
```

猫の画像をコピーして DrRacket に貼り付け、define で画像に名前を付けましょう。

Exercise 45. 「バーチャルキャット」の world プログラムを設計せよ。猫を左から右に継続的に動かす。cat-prog と呼び、猫の開始位置を消費すると仮定せよ。さらに、猫をクロックティックあたり3ピクセル動かせ。猫が右端で消えたら左端に再出現させる。modulo 関数について読むとよい。

Exercise 46. 少し異なる画像で猫のアニメーションを改善せよ：

```
(define cat2 <画像>)
```

Exercise 45 の描画関数を調整し、x座標が奇数かどうかで2つの猫画像のどちらかを使うようにせよ。HelpDesk で odd? を調べ、cond 式を使って猫画像を選択せよ。

Exercise 47. 「幸福ゲージ」を維持・表示する world プログラムを設計せよ。gauge-prog と呼び、幸福度は0から100（両端を含む）の数値であると合意せよ。

各クロックティックで幸福レベルは0.1減少する。ただし0を下回らない。下矢印キーが押されるたびに幸福度は1/5減少、上矢印が押されると1/3ジャンプする。

幸福レベルを表示するために、黒い枠のある塗りつぶしの赤い長方形を持つシーンを使え。幸福レベル0では赤いバーは消え、最大幸福レベルHではバーがシーン全体にわたるようにせよ。

Note. 十分な知識を得たら、gauge-prog を Exercise 45 の解と組み合わせる方法を説明する。その後、猫を助けることができるようになる。無視し続けると猫はますます不幸になる。撫でるとより幸せに、餌をやるとずっとずっと幸せになる。だからこそ、これらの最初の3章が教えてくれるよりもずっと多くの world プログラムの設計について知りたいと思う理由がわかるだろう。

注意：この章のすべての Racket コード、関数定義、check-expect、画像リテラル、big-bang 式などは原文のまま保持しています。翻訳は説明文、演習の記述、注記、設計プロセスの解説のみです。

原典: How to Design Programs, Second Edition - Chapter 3 “How to Design Programs” URL: https://htdp.org/2026-5-28/Book/part_one.html ## 4 区間、列挙、項目化 (Intervals, Enumerations, and Itemizations)

現時点では、情報をデータとして表現するための選択肢は、数値、文字列、画像、真偽値の4つしかありません。多くの問題に対してはこれで十分ですが、これら4つの基本データの集合だけでは対応しきれない複雑な問題も多く存在します。優れたソフトウェア設計者には、情報を表現するためのさらなる仕組みが必要です。

少なくとも、プログラムを設計する際には、これらの組み込みデータの集合に対して「制約」を課す方法を学ばねばなりません。制約を課す一つの方法は、データ集合からいくつかの具体的なデータだけをリストアップし、この問題においてはそれらしか使用しないと合意することです。このように要素をリストアップする方法を列挙 (Enumeration) と呼びますが、これは要素数が有限の場合にのみ有効です。一方で、「無限に」多くの要素を含む集合を扱うために、特定の条件を満たす要素の範囲を指定する区間 (Interval) を導入します。

[!NOTE] ここでいう「無限」とは、「要素数が多すぎて、一つひとつリストアップするのが完全に不可能である」という現実的な意味も含まれます。

列挙や区間を定義することは、異なる種類の要素を区別することを意味します。プログラムコード内でこれらを区別するには、条件分岐関数 (Conditional Functions)、すなわち引数の値に応じて異なる計算方法を選択する関数が必要です。「プロローグ：プログラムの仕方」や「ブール値と混ぜ合わせ」では、こうした条件分岐関数の書き方をいくつかの例で示しましたが、いずれも体系的な設計手法 (設計レシピ) は用いていませんでした。本章では、列挙、区間、そしてこれらを組み合わせた新しいデータ表現方法である項目化 (Itemization) の一般的な設計手法を解説します。

まず、cond 式の使い方を詳細に見直し、その後に列挙、区間、項目化の3つのデータ定義方法について説明します。列挙はそのクラスに属するすべてのデータを一つずつリストアップするものであり、区間はデータの範囲を指定するものです。最後の項目化は、定義の一部で範囲を指定し、別の部分で特定のデータを指定するという、両者を混ぜ合わせたものです。最後に、これらのデータ型に対する一般的な設計レシピ (設計戦略) を整理します。

4.1 条件式を使ったプログラミング (Programming with Conditionals)

「プロローグ：プログラムの仕方」で学んだ条件式を思い出してください。cond 式は本書で最も複雑な式の形であるため、その一般的な形式を詳しく見てみましょう。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2])
```

```
...
[ConditionExpressionN ResultExpressionN])
```

cond 式はキーワードである (cond で始まり、対応する) で終わります。キーワードに続いて、必要なだけの cond 行 (これを cond 節 と呼びます) を記述します。各 cond 節は、角括弧 [と] で囲まれた 2 つの式で構成されます (※丸括弧 () を使っても文法上問題ありませんが、視認性を高めるために角括弧を使うのが一般的です)。

条件式を使用した関数定義の例を見てみましょう：

```
(define (next traffic-light-state)
  (cond
    [(string=? "red" traffic-light-state) "green"]
    [(string=? "green" traffic-light-state) "yellow"]
    [(string=? "yellow" traffic-light-state) "red"])))
```

この例は、cond 式を使用することの便利さを示しています。多くの問題において、関数は複数の異なる状況を区別しなければなりません。cond 式を使用すれば、発生しうる可能性ごとに 1 つの cond 節を割り当てることができ、問題文で示された異なる状況とプログラムコードとの対応関係を読者に対して明確に示すことができます。

if 式 (「ブール値と混ぜ合わせ」を参照) との使い分けについての注意：if 式は「ある 1 つの状況と、それ以外のすべての状況」を区別するために使われます。そのため、3 つ以上の状況を区別する必要がある文脈には適していません。if 式は「A か B か」の二者択一を簡潔に表現する場合にのみ使用すべきです。したがって、データ定義から導き出される複数の明確に区別された状況をコードで表現する場合には、常に cond 式を使用します。

cond 式の条件が複雑になり、「その他のすべての場合」と表現したい場合があります。このような場合、cond 式の最後の節に限って else キーワードを使用することが許可されています。

```
(cond
  [ConditionExpression1 ResultExpression1]
  [ConditionExpression2 ResultExpression2]
  ...
  [else DefaultResultExpression])
```

もし最後以外の cond 節で else を使用すると、DrRacket の BSL はエラーを返します：

```
> (cond
  [(> x 0) 10]
  [else 20]
  [(< x 10) 30])
cond:found an else clause that isn't the last clause in its cond expression
```

文法的に正しくないプログラムコードは意味を持たないため、BSL は実行前にこれを拒否します。

ゲームプログラムの一部として、ゲーム終了時のスコアに応じて賞（アワード）を計算する関数 `reward` を設計するとします。関数のヘッダとデータ定義は以下の通りです：

```
; A PositiveNumber is a Number greater than/equal to 0.  
; PositiveNumber -> String  
; computes the reward level from the given score s
```

この関数を実装する2つのバリエーションを比較してみましょう：

左側の実装（3つの明確な条件を使用）：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [(< 20 s) "gold"])))
```

右側の実装（`else` 節を使用）：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [else "gold"])))
```

左側の実装では、3番目の条件として `(< 20 s)` を正しく記述するために、以下の計算が成り立つことをプログラマが頭の中で保証する必要があります：`* s` は非負の数値であること。`* (<= 0 s 10)` が `#false` であること。`* (and (< 10 s) (<= s 20))` もまた `#false` であること。

この例では計算は簡単に見えますが、複雑な条件分岐では境界値の計算ミスによってプログラムにバグ（不具合）が混入しやすくなります。したがって、最後のケースが他の条件の完全な「補集合（それ以外）」であることが明らかな場合は、右側のように `else` 節を使用する方が安全で優れています。

4.2 条件付きの計算 (Computing Conditionally)

DrRacket が `cond` 式をどのように評価するかをより正確に見ていきましょう。先ほどの `reward` 関数の定義を考えます：

```
(define (reward s)  
  (cond  
    [(<= 0 s 10) "bronze"]  
    [(and (< 10 s) (<= s 20)) "silver"]  
    [else "gold"])))
```

関数を呼び出す際、`cond` 式単体を見ただけではどの節が実行されるかを予測することはできません。これが関数の目的そのものです。関数は多様な入力（例えば 3, 12, 21 など）を受け取り、それぞれの入力に対して異なる振る舞いをします。入力データのバリエーションを区別して処理することが `cond` 式の役割です。

具体的な評価ステップを見てみましょう。まず、(reward 3) を呼び出した場合です：

```
(reward 3)
==
(cond
  [(<= 0 3 10) "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])
==
(cond
  [#true "bronze"]
  [(and (< 10 3) (<= 3 20)) "silver"]
  [else "gold"])
==
"bronze"
```

ここでは、最初の条件 ($\leq 0 \ 3 \ 10$) が #true に評価されたため、対応する結果式である "bronze" が cond 式全体の値となります。

次に、(reward 21) を呼び出した場合を見てみます：

```
(reward 21)
==
(cond
  [(<= 0 21 10) "bronze"]
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [#false "bronze"]
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
```

最初の条件が #false と評価されたため、その cond 節全体が評価対象から取り除かれ、次の節へと計算が進みます：

```
(cond
  [(and (< 10 21) (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and #true (<= 21 20)) "silver"]
  [else "gold"])
==
(cond
  [(and #true #false) "silver"]
  [else "gold"])
```

```

==
(cond
  [#false "silver"]
  [else "gold"])
==
(cond
  [else "gold"])
==
"gold"

```

このように、1 番目と 2 番目の条件が共に #false であったため、最後の else 節へと処理が移行し、結果として "gold" が得られます。

Exercise 48. DrRacket の定義領域に reward 関数の定義と (reward 18) を入力し、ステップ実行 (Stepper) を使用して評価のプロセスを確認せよ。

Exercise 49. cond 式は式の種類であるため、他の式の中にネスト (入れ子に) して記述することができます。

```
(- 200 (cond [(> y 200) 0] [else y]))
```

y の値が 100 の場合と 210 の場合について、Stepper を使ってこの式がどのように評価されるかを確認せよ。

また、図 20 に示す UFO/ロケットのアニメーションシーン描画関数 create-rocket-scene.v5 をリファクタリングし、cond 式をネストさせることで、place-image の記述が一度だけで済むように実装を書き換えよ。

```

(define WIDTH 100)
(define HEIGHT 60)
(define MTSCN (empty-scene WIDTH HEIGHT))
(define ROCKET <画像>)
(define ROCKET-CENTER-TO-TOP (- HEIGHT (/ (image-height ROCKET) 2)))

; 元の実装
(define (create-rocket-scene.v5 h)
  (cond
    [(<= h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 h MTSCN)]
    [(> h ROCKET-CENTER-TO-TOP) (place-image ROCKET 50 ROCKET-CENTER-TO-TOP
      MTSCN)]))

```

4.3 列挙 (Enumerations)

すべての文字列が任意に使えるわけではありません。例えば、マウスイベントをプログラムに通知する文字列は以下の 6 つに制限されています：

```

; A MouseEvent is one of these Strings:
; - "button-down"
; - "button-up"
; - "drag"
; - "move"

```



```
; - "enter"
; - "leave"
```

このように、取りうるすべての具体的なデータを網羅して定義するデータ型を列挙 (Enumeration) と呼びます。日常のプログラム設計において、列挙型は非常によく現れます。例えば、信号機の状態を以下のように定義できます：

```
; A TrafficLight is one of the following Strings:
; - "red"
; - "green"
; - "yellow"
; interpretation: the three strings represent the three possible states of a
traffic light
```

列挙データ型を消費する関数の設計は非常にシンプルです。入力データがケースバイケースで列挙されている場合、関数側でもそれぞれのケースを `cond` 節で区別して処理します。信号機の状態を受け取って、次の信号機の状態を返す関数 `traffic-light-next` は以下のように設計できます：

```
; TrafficLight -> TrafficLight
; yields the next state given current state s
(check-expect (traffic-light-next "red") "green")

(define (traffic-light-next s)
  (cond
    [(string=? "red" s) "green"]
    [(string=? "green" s) "yellow"]
    [(string=? "yellow" s) "red"])))
```

`TrafficLight` の定義が3つのデータ要素で構成されているため、`traffic-light-next` 関数もまた自然に3つの `cond` 節で分岐を行います。

Exercise 50. 上記の `traffic-light-next` 関数の定義を DrRacket にコピー&ペーストして RUN すると、テストが不十分なため、いくつかの `cond` 節がハイライト表示 (テスト未通過) されます。すべての節がカバーされるようにテストケースを追加せよ。

Exercise 51. 信号機のアニメーションをシミュレートする `big-bang` プログラムを設計せよ。信号機の状態を適切な色の円で描画し、クロックティックごとに状態 (色) が遷移するようにせよ。最も適切な初期状態について検討せよ。

列挙型の本質は、データ集合を「有限個の具体的なデータ片」として定義することです。もう一つの重要な例を紹介します：

```
; A lString is a String of length 1, including:
; - "\" (the backslash),
; - " " (the space bar),
; - " " (tab),
; - "" (return), and
```



```
; - "␣" (backspace).
; interpretation: represents keys on the keyboard
```

BSL では、キーボードの入力イベント (KeyEvent) を以下のように表現します：

```
; A KeyEvent is one of:
; - lString
; - "left"
; - "right"
; - "up"
; - "down"
; - ...
```

この定義に従うと、キーイベントを処理する関数 (キーイベントハンドラ) の設計テンプレートは自然と以下ようになります：

```
; WorldState KeyEvent -> WorldState
(define (handle-key-events w ke)
  (cond
    [(= (string-length ke) 1) ...]
    [(string=? "left" ke) ...]
    [(string=? "right" ke) ...]
    [(string=? "up" ke) ...]
    [(string=? "down" ke) ...]
    [else ...]))
```

Exercise 52. キーボードの左右の矢印キーが押されたときに、赤いドットを左右に移動させるキーイベントハンドラ `keh` を設計せよ。不要なキーイベントは `else` 節で無視すること。図 21 に示す以下のコードを参考にせよ。

```
; A Position is a Number.
; interpretation: distance between the left margin and the ball
; Position KeyEvent -> Position
; computes the next location of the ball
```

```
(check-expect (keh 13 "left") 8)
(check-expect (keh 13 "right") 18)
(check-expect (keh 13 "a") 13)
```

```
(define (keh p k)
  (cond
    [(string=? "left" k) (- p 5)]
    [(string=? "right" k) (+ p 5)]
    [else p]))
```

4.4 区間 (Intervals)

UFO が降下する様子を描画するプログラムを考えます (図 22 を参照)。

```
; A WorldState is a Number.
; interpretation: number of pixels between the top and the UFO
(define WIDTH 300)
```

```

(define HEIGHT 100)
(define CLOSE (/ HEIGHT 3))
(define MTSCN (empty-scene WIDTH HEIGHT))
(define UFO (overlay (circle 10 "solid" "green") (rectangle 30 4 "solid"
"green")))

; WorldState -> WorldState
(define (main y0)
  (big-bang y0
    [on-tick nxt]
    [to-draw render]))

; WorldState -> WorldState
(define (nxt y) (+ y 3))

; WorldState -> Image
(define (render y)
  (place-image UFO (/ WIDTH 2) y MTSCN))

```

このUFOシミュレーションにおいて、UFOの高さ（位置）に応じて以下のようなステータス画面を表示させたいとします：* UFOの高さが画面上部の3分の1（CLOSE）より上にあるときは "descending"（降下中）。* それを超えて下部（HEIGHT）までの間にあるときは "closing in"（接近中）。* 画面の底（HEIGHT）に達したときは "landed"（着陸）。

この場合、状態を表す数値は無限に存在するため、単純に個々の値を列挙することは不可能です。代わりに、境界値を用いて数値の範囲を切り分ける区間（Intervals）を使用します。

区間は、境界値（左端と右端）によって定義されます。境界値自体が範囲に含まれる場合を「閉じた（closed）」、含まれない場合を「開いた（open）」と呼びます。数学の表記に倣い、閉じた境界にはブラケット [や]、開いた境界にはペアレントシス (や) を用いて表現します：* [3, 5]：閉区間（3以上5以下）* (3, 5]：左開・右閉区間（3より大きく5以下）* [3, 5)：左閉・右开区間（3以上5未満）* (3, 5)：开区間（3より大きく5未満）

これを用いると、世界の状態 WorldState は以下の3つの区間に分類できます：1. 0 から CLOSE の間（降下中）2. CLOSE から HEIGHT の間（接近中）3. HEIGHT 以上（着陸）

これをプログラムで表現する場合、境界値がどの区間に属するかという「重複」を避ける必要があります。BSLの cond 条件式は順番に評価されるため、この性質を利用して以下のように記述できます：

```

; WorldState -> Image
(define (render/status y)
  (cond
    [(<= 0 y CLOSE)
     (place-image (text "descending" 11 "green") 10 10 (render y))]
    [(and (< CLOSE y) (<= y HEIGHT))

```

```

    (place-image (text "closing in" 11 "orange") 10 10 (render y)))
  [(> y HEIGHT)
   (place-image (text "landed" 11 "red") 10 10 (render y))])

```

UF0 のアニメーションプログラムをこのステータス表示付きのバージョンにするには、main 関数で [to-draw render/status] を呼び出すように変更します。

UF0 の位置情報や表示位置を一箇所で管理するために、UF0 の座標やテキスト表示位置の「コピー」を避ける実装を設計するのが賢明です。図 24 のように、cond 式自体を place-image の引数として入れ子にすることができます：

```

(define (render/status y)
  (place-image
   (cond
    [(<= 0 y CLOSE) (text "descending" 11 "green")]
    [(and (< CLOSE y) (<= y HEIGHT)) (text "closing in" 11 "orange")]
    [else (text "landed" 11 "red")])
   20 20
   (render y)))

```

4.5 項目化 (Itemizations)

列挙型は具体的な有限のデータ片をリストアップし、区間は無限に存在する数値の範囲を指定します。これら両方を混在させて定義するデータ型を項目化 (Itemizations) と呼びます。

例えば、string->number 関数の出力結果を考えてみましょう。この関数は、文字列が数値に変換可能な場合は数値を返し、不可能な場合は #false を返します。この結果を表すデータ型 NorF は以下のように定義できます：

```

; An NorF is one of:
; - #false
; - a Number

```

これは、具体的なデータ片である #false と、無限のデータクラスである Number を組み合わせた項目化の典型例です。この NorF を消費する関数の設計は、やはりそれぞれのケース (項目) に対応する cond 節を持つ形になります：

```

; NorF -> Number
; adds 3 to the given number; 3 otherwise
(check-expect (add3 #false) 3)
(check-expect (add3 0.12) 3.12)

(define (add3 x)
  (cond
   [(false? x) 3]
   [else (+ x 3)]))

```

もう少し実用的な例として、ロケットの打ち上げカウントダウンをシミュレートするプログラムを設計します。

```
; An LRCD (for launching rocket countdown) is one of:
; - "resting"
; - a Number between -3 and -1
; - a NonnegativeNumber
; interpretation:
;   "resting" represents a grounded rocket.
;   a number in [-3, -1] denotes the countdown phase.
;   a nonnegative number denotes the height of the rocket in flight.
```

このデータ型に対するシミュレーションプログラムを以下のように構成します： * 定数定義：

```
(define HEIGHT 300)
(define WIDTH 100)
(define YDELTA 3)
(define BACKG (empty-scene WIDTH HEIGHT))
(define ROCKET (rectangle 5 30 "solid" "red"))
(define CENTER (/ (image-height ROCKET) 2))
```

• 描画関数 **show**:

```
; LRCD -> Image
(define (show x)
  (cond
    [(string? x)
     (place-image ROCKET 10 (- HEIGHT CENTER) BACKG)]
    [(<= -3 x -1)
     (place-image (text (number->string x) 20 "red")
                   10 (* 3/4 WIDTH)
                   (place-image ROCKET 10 (- HEIGHT CENTER) BACKG))]
    [(>= x 0)
     (place-image ROCKET 10 (- x CENTER) BACKG))])
```

• キーイベントハンドラ **launch**:

```
; LRCD KeyEvent -> LRCD
(define (launch x ke)
  (cond
    [(string? x) (if (string=? " " ke) -3 x)]
    [(<= -3 x -1) x]
    [(>= x 0) x]))
```

• 時間経過ハンドラ **fly** (カウントダウンおよび飛行のシミュレーション) :

```
; LRCD -> LRCD
(define (fly x)
  (cond
    [(string? x) x]
    [(<= -3 x -1) (if (= x -1) HEIGHT (+ x 1))]
    [(>= x 0) (- x YDELTA)]))
```

• メインプログラム実行:

```
(define (main2 s)
  (big-bang s
    [to-draw show]
    [on-key launch]
    [on-tick fly 1])) ; クロックティックは1秒ごと
```

Exercise 53～57. 以上のカウントダウン付きロケットプログラムを実際に DrRacket 上で構築・実行し、終了処理 (stop-when) の追加などの拡張を試みよ。

4.6 項目化を用いた設計 (Designing with Itemizations)

これまでに学んだ内容を基に、「From Functions to Programs」で紹介した設計レシピを拡張し、列挙、区間、項目化を扱う関数を体系的に設計するためのステップを定義します。

以下の消費税計算を例に、ステップを確認します： > 安価な商品 (1,000 ドル未満) には税金がかかりません。 > 高級品 (10,000 ドル超) には 8% の税率が適用されます。 > それらの中間の商品には 5% の税金がかかります。

1. データ定義 (Data Definition) 問題文で区別されている状況をカバーする、重複のない明確なデータ定義を記述します。

```
; A Price falls into one of three intervals:
; - 0 through 1000 (exclusive)
; - 1000 through 10000 (exclusive)
; - 10000 and above.
```

2. 署名、目的文、ヘッダ (Signature, Purpose, Header)

```
; Price -> Number
; computes the amount of tax charged for p
(define (sales-tax p) 0)
```

3. 具体的な例 (Examples) それぞれの区間、および境界値からテストデータを抽出します (境界値の扱いについて専門家や仕様書に確認することが重要です)。

```
(check-expect (sales-tax 537) 0)
(check-expect (sales-tax 1000) (* 0.05 1000))
(check-expect (sales-tax 12017) (* 0.08 12017))
```

4. テンプレート (Template) データ定義の項目 (サブクラス) の数と同じ数の cond 節を持つ条件分岐のアウトラインを作成します。

```
(define (sales-tax p)
  (cond
    [(and (<= 0 p) (< p 1000)) ...]
    [(and (<= 1000 p) (< p 10000)) ...]
    [(>= p 10000) ...]))
```

5. 定義の完成 (Definition) 各節の空欄 ... を、パラメータを用いた具体的な計算式で埋めます。

```
(define (sales-tax p)
  (cond
    [(and (<= 0 p) (< p 1000)) 0]
    [(and (<= 1000 p) (< p 10000)) (* 0.05 p)]
    [(>= p 10000) (* 0.08 p)]))
```

6. テスト (Testing) テストを実行し、すべての `cond` 節がカバーされ、期待通りの値が返ることを確認します。

4.7 有限状態世界 (Finite State Worlds)

信号機のように、「有限個の状態」と「状態間の遷移ルール」によって定義されるシステムを有限状態機械 (Finite State Machine: FSM) または有限状態オートマトン (Finite State Automaton: FSA) と呼びます。

信号機のシミュレーションプログラムは、以下の要素で構成できます： * 状態の定義: "red", "green", "yellow" * 時間経過による遷移関数 `tl-next`: racket ; TrafficLight -> TrafficLight (define (tl-next cs) (cond [(string=? "red" cs) "green"] [(string=? "green" cs) "yellow"] [(string=? "yellow" cs) "red"]))) * 描画関数 `tl-render`: 状態に応じて適切な色のライトを表示する画像を生成する。

Exercise 59~61. 信号機の有限状態シミュレーションプログラムを完成させよ。

自動ドアクローザーのシミュレーション

自動ドアは以下の状態と遷移ルールを持ちます： * 状態 (`DoorState`): * LOCKED (施錠中) * CLOSED (閉扉中・未施錠) * OPEN (開扉中) * イベントルール: * LOCKED 状態で "u" キーを押すと CLOSED になる (解錠)。 * CLOSED 状態で "l" キーを押すと LOCKED になる (施錠)。 * CLOSED 状態でスペースキー " " を押すと OPEN になる (開扉)。 * OPEN 状態からは、クロックティック (時間経過) により自動的に CLOSED に戻る。

このシミュレーションプログラムは以下のように設計できます：

```
(define LOCKED "locked")
(define CLOSED "closed")
(define OPEN "open")

; DoorState -> DoorState
; closes an open door over the period of one tick
(define (door-closer state-of-door)
  (cond
    [(string=? LOCKED state-of-door) LOCKED]
    [(string=? CLOSED state-of-door) CLOSED]
    [(string=? OPEN state-of-door) CLOSED]))

; DoorState KeyEvent -> DoorState
; turns key event k into an action on state s
(define (door-action s k)
```

```

(cond
  [(and (string=? LOCKED s) (string=? "u" k)) CLOSED]
  [(and (string=? CLOSED s) (string=? "l" k)) LOCKED]
  [(and (string=? CLOSED s) (string=? " " k)) OPEN]
  [else s]))

; DoorState -> Image
; translates the state s into a large text image
(define (door-render s)
  (text s 40 "red"))

; DoorState -> DoorState
(define (door-simulation initial-state)
  (big-bang initial-state
    [on-tick door-closer 3] ; 3秒ごとにドアが閉まる
    [on-key door-action]
    [to-draw door-render]))

```

Exercise 62. ドアシミュレータを DrRacket で実行し、動作を確認せよ。

5 構造の追加 (Adding Structure)

(今後の翻訳作業のためのプレースホルダー)

6 項目化と構造 (Itemizations and Structures)

(今後の翻訳作業のためのプレースホルダー)

7 まとめ (Summary)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 1: Beginning Student Language

(今後の翻訳作業のためのプレースホルダー)

II 任意サイズのデータ (Arbitrarily Large Data)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 2: Quote, Unquote

(今後の翻訳作業のためのプレースホルダー)

III 抽象化 (Abstraction)

導入：抽象化の必要性

多くのデータ定義と関数定義は似ています。例えば、String のリストの定義は Number のリストの定義と、データクラスの名前と「String」「Number」という単語の 2 箇所しか異

なりませ。同様に、String のリストから特定の文字列を探す関数は、Number of lists から特定の数を探す関数とほとんど区別が付きません。

経験から、このような類似性は問題を引き起こすことがわかっています。類似性は、プログラマがコードを（物理的または精神的に）コピーすることから生じます。プログラマが別の問題に直面したとき、既存の解決策をコピーして新しい問題に合わせて修正します。この行動は「本物の」プログラミングだけでなく、スプレッドシートや数学的モデリングの世界でも見られます。

しかし、コードのコピーは以下のような問題を引き起こします： - ミスのコピー - 同じ修正を多くのコピーに適用する必要性 - 基礎データ定義が変更されたときにすべてのコピーを探して修正するコスト

このプロセスは高価でエラーが発生しやすく、プログラミングチームに不必要なコストを課します。

良いプログラマは、プログラミング言語が許す限り類似性を排除しようとします。

「プログラムはエッセイのようなものです。最初のバージョンは下書きであり、下書きには編集が必要です。」

「排除する」とは、プログラマがプログラムの最初のドラフトを書き留め、類似性（および他の問題）を見つけ、それらを取り除くことを意味します。最後のステップでは、抽象化を行うか、既存の（抽象化された）関数を使用します。このプロセスを何度か繰り返すことで、プログラムを満足のいく形にすることがよくあります。

このパートの前半では、関数とデータ定義の類似性を抽象化する方法を示します。プログラマはこのプロセスの結果も「抽象化」と呼びます。後半は、既存の抽象化の使用と、このプロセスを容易にする新しい言語要素についてです。このパートの例はリストの領域から取られていますが、考え方は普遍的に適用可能です。

14 類似性はどこにでもある (Similarities Everywhere)

Arbitrarily Large Data の演習（の一部）を解いた人は、多くの解決策が似ていることを知っているでしょう。実際、類似性は、ある問題の解決策をコピーして次の問題の解決策を作成する誘惑に駆られるかもしれません。

しかし、thou shall not steal code（コードを盗んてはならない）、たとえ自分のコードであってもです。代わりに、類似したコード片を抽象化しなければなりません。この章では、その抽象化の方法を教えます。

私たちの手段は「Intermediate Student Language」（ISL）に特有のものです。

DrRacket で、「Language」メニューの「How to Design Programs」サブメニューから「Intermediate Student」を選択してください。

ほとんどすべての他のプログラミング言語も同様の手段を提供しています。オブジェクト指向言語では、追加の抽象化メカニズムが見つかるかもしれません。いずれにせよ、これらのメカニズムは本章で述べる基本的な特性を共有しており、ここで説明する設計の考え方は他の文脈でも適用されます。

14.1 関数における類似性 (Similarities in Functions)

設計レシピは、関数の基本的な組織を決定します。なぜなら、テンプレートは関数の目的に関係なくデータ定義から作成されるからです。したがって、同じ種類のデータを消費する関数は似ているのは当然です。

以下は 2 つの非常に似た関数です (図 86) :

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (cond
    [(empty? l) #false]
    [else
     (or
      (string=? (first l) "dog")
      (contains-dog? (rest l)))]))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (cond
    [(empty? l) #false]
    [else
     (or
      (string=? (first l) "cat")
      (contains-cat? (rest l)))]))
```

図 86: 2 つの類似した関数

これらの関数はほぼ区別が付きません。各々は文字列のリストを消費し、本体は 2 つの節を持つ cond 式で構成されます。入力为空リストなら #false を返し、or を使って最初の要素が目的の文字列かどうかを調べ、そうでなければ再帰的に残りを調べます。唯一の違いは比較する文字列です (図 86 の “dog” と “cat”)。

良いプログラマは、このような密接に関連する関数を複数定義するのを避けます。代わりに、探す文字列を追加の引数として受け取る 1 つの汎用関数を定義します :

```
; String Los -> Boolean
; determines whether l contains the string s
(define (contains? s l)
  (cond
    [(empty? l) #false]
    [else
```

```
(or (string=? (first l) s)
    (contains? s (rest l))))))
```

これで contains-dog? と contains-cat? は1行で定義できます：

```
; Los -> Boolean
; does l contain "dog"
(define (contains-dog? l)
  (contains? "dog" l))

; Los -> Boolean
; does l contain "cat"
(define (contains-cat? l)
  (contains? "cat" l))
```

図 87: 2つの類似した関数、再考

これが関数的抽象化 (Functional Abstraction) です。異なるバージョンの関数を抽象化することは、プログラムから類似性を排除する一つの方法であり、類似性を排除することで、長期にわたってプログラムを健全に維持することが容易になります。

Exercise 235. contains? を使って “atom”, “basic”, “zoo” を探す関数を定義せよ。

Exercise 236. 次の2つの関数のテストスイートを作成せよ：

```
; Lon -> Lon
; adds 1 to each item on l
(define (add1* l)
  (cond
    [(empty? l) '()]
    [else
     (cons (add1 (first l)) (add1* (rest l)))]))

; Lon -> Lon
; adds 5 to each item on l
(define (plus5 l)
  (cond
    [(empty? l) '()]
    [else
     (cons (+ (first l) 5) (plus5 (rest l)))]))
```

これらを抽象化せよ。抽象化した関数を使って上記2つを1行で再定義し、テストで確認せよ。最後に、各数から2を引く関数を設計せよ。

14.2 異なる類似性 (Different Similarities)

contains-dog? と contains-cat? のケースでは抽象化は簡単に見えました。2つの関数定義を比較し、リテラル文字列を関数パラメータに置き換え、元の関数が抽象関数を使って簡単に定義できることを確認するだけでした。この種の抽象化は非常に自然なため、本書の前半2つのパートでも特に断ることなく登場していました。

本節では、これと同じ原理が、より強力な抽象化の形態をどのように生み出すかを示します。図 88 を見てください。どちらの関数も数値のリストと閾値を消費します。左側は閾値未満のすべての数値のリストを生成し、右側は閾値を超えるすべての数値のリストを生成します。

```
; Lon Number -> Lon
; select those numbers on l
; that are below t
(define (small l t)
  (cond
    [(empty? l) '()]
    [else
     (cond
       [(< (first l) t) (cons (first l) (small (rest l) t))]
       [else (small (rest l) t))]]))

; Lon Number -> Lon
; select those numbers on l
; that are above t
(define (large l t)
  (cond
    [(empty? l) '()]
    [else
     (cond
       [(> (first l) t) (cons (first l) (large (rest l) t))]
       [else (large (rest l) t))]]))
```

図 88: さらに 2 つの類似した関数

この 2 つの関数は、与えられたリストの数値が結果の一部になるべきかどうかを決定する「比較演算子」の 1 箇所だけが異なります。左側の関数は `<` を使用し、右側は `>` を使用しています。それ以外は、関数名を除いて 2 つの関数は同一です。

最初の例に従って、追加のパラメータを使ってこれら 2 つの関数を抽象化しましょう。今回は、追加のパラメータは文字列ではなく、比較演算子（関数）を表します：

```
(define (extract R l t)
  (cond
    [(empty? l) '()]
    [else
     (cond
       [(R (first l) t)
        (cons (first l) (extract R (rest l) t))]
       [else
        (extract R (rest l) t))]))
```

この新しい関数を適用するには、3 つの引数を指定する必要があります：2 つの数値を比較する関数 `R`、数値のリスト `l`、そして閾値 `t` です。この関数は、`(R i t)` が `#true` に評価される `l` 内のすべての要素 `i` を抽出します。

[!NOTE] ここで、この定義が意味をなすのか疑問に思うかもしれません。私たちは何の苦労もなく、「関数を消費する関数」を作成しました。微分演算や不定積分を学んだことがあるなら、これらが関数を消費して関数を生成する関数であることを知っているでしょう。しかし、本コースではその知識を仮定しません。私たちの単純な学習言語 ISL はこのような関数をサポートしており、このような関数を定義することは優れたプログラマの最も強力なツールの 1 つです。

テストを実行すると、`(extract < 1 t)` が `(small 1 t)` と同じ結果を計算することがわかります：

```
(check-expect (extract < '() 5) (small '() 5))
(check-expect (extract < '(3) 5) (small '(3) 5))
(check-expect (extract < '(1 6 4) 5) (small '(1 6 4) 5))
```

同様に、`(extract > 1 t)` は `(large 1 t)` と同じ結果を生成します。つまり、元の 2 つの関数を次のように再定義できます：

```
; Lon Number -> Lon
(define (small-1 l t) (extract < l t))

; Lon Number -> Lon
(define (large-1 l t) (extract > l t))
```

重要な洞察は、`small-1` と `large-1` が 1 行で定義できることだけではありません。`extract` のような抽象関数があれば、他の場所でも有効に活用できます：
* `(extract = 1 t)`：`l` 内の `t` と等しい数値をすべて抽出します。
* `(extract <= 1 t)`：`l` 内の `t` 以下の数値をすべて抽出します。
* `(extract >= 1 t)`：`l` 内の `t` 以上の数値をすべて抽出します。

実際、`extract` の第 1 引数は ISL の定義済み演算である必要はありません。2 つの引数を消費して真偽値を生成する任意の関数を使用できます。次の例を考えてみましょう：

```
; Number Number -> Boolean
; is the area of a square with side x larger than c
(define (squared>? x c)
  (> (* x x) c))
```

すなわち、`squared>?` は平方 $x^2 > c$ が成り立つかどうかを検査し、`extract` で使用できます：

```
(extract squared>? (list 3 4 5) 10)
```

この適用により、`(list 3 4 5)` の中で平方した値が 10 より大きい数値（この場合は 4 と 5）が抽出されます。

Exercise 237. DrRacket で `(squared>? 3 10)` と `(squared>? 4 10)` を評価せよ。`(squared>? 5 10)` はどうなるか。

抽象化された関数定義は、元の関数よりも便利になり得ることがわかりました。たとえば、contains? は contains-dog? や contains-cat? よりも便利であり、extract は small や large よりも便利です。抽象化のこれらの利点は、文書作成、スプレッドシート、小さなアプリ、大規模な産業プロジェクトなど、あらゆるレベルのプログラミングで得られます。

また、抽象化のもう1つの重要な側面は、これらの関数に対して単一の制御点（Single Point of Control）を持てることです。抽象関数に間違いがあることが判明した場合、その定義を修正するだけで、他のすべての定義も修正されます。同様に、抽象関数の計算を高速化する方法を見つければ、この関数を使用して定義されたすべての関数が追加の労力なしに改善されます。

```
; Nelon -> Number
; determines the smallest number on l
(define (inf l)
  (cond
    [(empty? (rest l)) (first l)]
    [else
     (if (< (first l) (inf (rest l)))
         (first l)
         (inf (rest l))))]))

; Nelon -> Number
; determines the largest number on l
(define (sup l)
  (cond
    [(empty? (rest l)) (first l)]
    [else
     (if (> (first l) (sup (rest l)))
         (first l)
         (sup (rest l))))]))
```

図 89: リスト内の最小値 (inf) と最大値 (sup) の探索

Exercise 238. 図 89 の 2 つの関数を 1 つの関数に抽象化せよ。どちらも空でない数値のリスト (Nelon) を消費し、1 つの数値を生成します。左側はリスト内の最小値を生成し、右側は最大値を生成します。

抽象関数を用いて inf-1 と sup-1 を定義せよ。以下の 2 つのリストでテストせよ：

```
(list 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 9 8 7 6 5 4 3 2 1)
(list 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25)
```

なぜこれらの関数は長いリストの一部で遅くなるのか。元の関数を min (2 つの数値から小さい方を選択) および max (大きい方を選択) を使って修正せよ。その後、再度抽象化し、inf-2 と sup-2 を定義して、同じ入力でテストせよ。なぜこれらのバージョンははるかに高速になるのか。

14.3 データ定義における類似性 (Similarities in Data Definitions)

ここで、次の2つのデータ定義を詳しく見てみましょう：

```
; An Lon (List-of-numbers) is one of:  
; - '()  
; - (cons Number Lon)  
  
; An Los (List-of-strings) is one of:  
; - '()  
; - (cons String Los)
```

左側は数値のリストを導入し、右側は文字列のリストを記述しています。そして、これら2つのデータ定義は非常に似ています。類似した関数と同様に、2つのデータ定義は異なる名前を使用していますが、名前は任意であるためこれは重要ではありません。唯一の本当の違いは、2番目の節の `cons` の最初の位置にあり、リストがどのような種類の要素を含むかを指定している点です。

この1つの違いを抽象化するために、データ定義を関数であるかのように扱います。パラメータを導入し、異なる型（要素）を参照していた場所にこのパラメータを使用します：

```
; A [List-of ITEM] is one of:  
; - '()  
; - (cons ITEM [List-of ITEM])
```

このような抽象的なデータ定義をパラメータ化されたデータ定義 (Parametric Data Definitions) と呼びます。大雑把に言えば、パラメータ化されたデータ定義は、関数が特定の値から抽象化するのと同様に、特定のデータコレクションへの参照から抽象化します。

問題は、もちろん、これらのパラメータがどのような範囲をとるかです。関数のパラメータが「未知の値」を表すのに対し、データ定義のパラメータは「データのクラス全体(型)」を表します。パラメータ化されたデータ定義に具体的なデータコレクションの名前を割り当てるプロセスをインスタンス化 (Instantiation) と呼びます。以下は `List-of` 抽象化のサンプルインスタンス化です： * `[List-of Number]` と書くとき、`ITEM` が `Number` を表すため、これは `List-of-numbers` の別名になります。 * 同様に、`[List-of String]` は `List-of-strings` と同じデータクラスを定義します。 * 以下のような在庫レコード `IR` のクラスを定義した場合： `racket (define-struct IR [name price]) ; An IR is a structure: (make-IR String Number) [List-of IR]` は在庫レコードのリストの名前になります。

慣例として、データ定義のパラメータには大文字の名前（例：`ITEM`, `x`, `y`）を使用します。

この抽象データ定義が正当であることを確認するには、パラメータ `ITEM` に実際のデータ定義名（たとえば `Number`）を代入し、自己参照（再帰）を展開してみます。得られる定義は従来の数値リストの定義と完全に一致します。

2番目の例として、次の構造体定義から始めましょう：

```
(define-struct point [hori veri])
```

この構造体タイプを使用する 2 つの異なるデータ定義を示します：

```
; A Pair-boolean-string is a structure:  
;   (make-point Boolean String)
```

```
; A Pair-number-image is a structure:  
;   (make-point Number Image)
```

この場合、データ定義は 2 箇所異なります。hori フィールドに対応する違いと、veri フィールドに対応する違いです。したがって、抽象データ定義を作成するには 2 つのパラメータを導入する必要があります：

```
; A [CP H V] is a structure:  
;   (make-point H V)
```

ここで、H は hori フィールドのデータコレクション用のパラメータであり、V は veri フィールドに表示されるデータコレクション用のパラメータです。

2 つのパラメータを持つデータ定義をインスタンス化するには、データコレクションの 2 つの名前が必要です。パラメータに Number と Image を使用すると、数値と画像を組み合わせたポイント構造体のコレクションを記述する [CP Number Image] が得られます。対照的に、[CP Boolean String] は真偽値と文字列をポイント構造体で組み合わせます。

Exercise 239. 2 つの要素のリストは、ISL プログラミングで頻繁に使用されるもう 1 つのデータ形式です。以下は、2 つのパラメータを持つデータ定義です：

```
; A [List X Y] is a list:  
;   (cons X (cons Y '()))
```

この定義をインスタンス化して、以下のデータクラスを記述せよ：1. 数値のペア (Number と Number) 2. 数値と lString のペア 3. 文字列と真偽値のペア

また、これら 3 つのデータ定義のそれぞれについて、具体的な例を 1 つずつ作成せよ。

パラメータ化されたデータ定義が得られたら、それらを組み合わせて大きな効果を得ることができます。次の定義を考えてみましょう：

```
; [List-of [CP Boolean Image]]
```

最も外側の表記は [List-of ...] であり、これはリストを扱っていることを意味します。このリストがどのようなデータを含むかという問いに答えるには、内部の式を調べる必要があります：[CP Boolean Image]。これは真偽値と画像をポイント内で組み合わせたものです。含意として、上記は真偽値と画像を組み合わせたポイントのリストです。同様に、

```
; [CP Number [List-of Image]]
```

は、1 つの数値と画像のリストを組み合わせた CP のインスタンス化です。

Exercise 240. 奇妙ですが類似した次の 2 つのデータ定義があります：


```
; An LStr is one of:
; - String
; - (make-layer LStr)

; An LNum is one of:
; - Number
; - (make-layer LNum)
```

どちらのデータ定義も、次の構造体定義に依存しています：

```
(define-struct layer [stuff])
```

両者について具体例を作成せよ。2つを抽象化したデータ定義を作成せよ。そして、抽象定義をインスタンス化して元の定義を復元せよ。

Exercise 241. NEList-of-temperatures と NEList-of-Booleans の定義を比較せよ。そして、空でないリストを表す抽象データ定義 NEList-of を策定せよ。

Exercise 242. 以下は、もう1つのパラメータ化されたデータ定義です：

```
; A [Maybe X] is one of:
; - #false
; - X
```

これらのデータ定義を解釈せよ：[Maybe String], [Maybe [List-of String]], [List-of [Maybe String]]。

次の関数署名は何を意味するか：

```
; String [List-of String] -> [Maybe [List-of String]]
; returns the remainder of los starting with s
; #false otherwise
(check-expect (occurs "a" (list "b" "a" "d" "e"))) (list "d" "e"))
(check-expect (occurs "a" (list "b" "c" "d"))) #false)
(define (occurs s los) los)
```

設計レシピの残りのステップを実行せよ。

14.4 関数は値である (Functions Are Values)

本パートの関数は、プログラムの評価に関する私たちの理解を拡張します。関数が数値だけでなく、文字列や画像を消費することは理解しやすいでしょう。構造体やリストは少し拡張が必要ですが、結局のところ有限の「もの」です。しかし、関数を消費する関数は奇妙に思えます。実際、このアイデア自体が最初のインターメッツォ (BSL) の規則に2つの方法で違反しています：1. プリミティブや関数の名前が、適用式の引数として使用されている。2. パラメータが、適用式の「関数位置」で使用されている。

問題を詳しく説明すると、ISLの文法がBSLとどのように異なるかがわかります。第1に、式言語は定義内の関数名とプリミティブ演算の名前を含めるべきです。第2に、適用式の最初の位置は、関数名やプリミティブ演算以外のもの (変数や関数のパラメータなど) を許容しなければなりません。

文法への変更は評価規則への変更を要求するようになりますが、変更されるのは「値の集合」だけです。具体的には、関数を関数の引数として受け入れるために、最も単純な変更は「関数やプリミティブ演算もまた値である」とすることです。

Exercise 243. DrRacket の定義領域に以下が含まれていると仮定します：

```
(define (f x) x)
```

以下の式のうち、値（値の定義を満たすもの）を特定せよ： 1. (cons f '()) 2. (f f) 3. (cons f (cons 10 (cons (f 10) '())))

なぜそれらが値である（または値でない）のかを説明せよ。

Exercise 244. 次の記述が ISL において合法（文法的に正しい）である理由を論じよ： 1. (define (f x) (x 10)) 2. (define (f x) (x f)) 3. (define (f x y) (x 'a y 'b))

あなたの推論を説明せよ。

Exercise 245. function=at-1.2-3-and-5.775? 関数を設計せよ。数値から数値への 2 つの関数が与えられたとき、この関数は、1.2、3、および -5.775 に対して 2 つの関数が同じ結果を生成するかどうかを判定します。

数学者は、同じ入力を与えられたときに同じ結果を計算する場合、2 つの関数は等しいと言います。数値から数値への 2 つの関数が等しいかどうかを判定する一般関数 function=? を定義することは可能でしょうか。もし可能なら定義せよ。不可能なら、その理由を説明し、これが「簡単に定義できるアイデアであるにもかかわらず、その関数を定義できない最初の例」に遭遇したことを意味するインプリケーションを考慮せよ。

14.5 関数を使った計算 (Computing with Functions)

BSL+ から ISL への移行により、関数を引数として使用し、適用式の最初の位置にパラメータ名を使用できるようになりました。DrRacket はこれらの位置にある名前を他の場所と同様に処理しますが、当然ながら結果として関数を期待します。驚くべきことに、代数の法則を単純に適用するだけで、ISL プログラムを評価することができます。

extract の評価がどのように機能するかを見てみましょう。明らかに、

```
(extract < '() 5) == '()
```

が成り立ちます。「Intermezzo 1」の代入モデルを使用して、関数の本体で計算を続けることができます。パラメータ R, l, t が、引数 <, '(), 5 にそれぞれ置き換えられます。ここからは、条件式の評価から始まるプレーンな計算です：

```
==
(cond
  [(empty? '()) '()]
  [else
   (cond
     [(< (first '()) 5)
      (cons (first '()) (extract < (rest '()) 5))]
     [else
      (extract < (rest '()) 5)]]])
```

```

      [else (extract < (rest '()) 5)]]])
==
(cond
  [#true '()]
  [else
    (cond
      [(< (first '()) 5)
        (cons (first '()) (extract < (rest '()) 5))]
      [else (extract < (rest '()) 5)]]])
== '()

```

次に、要素が1つのリストを考えます：

```
(extract < (cons 4 '()) 5)
```

リストの唯一の要素は 4 であり、(< 4 5) は真であるため、結果は (cons 4 '()) になるはずです。評価の最初のステップは次のようになります：

```

(extract < (cons 4 '()) 5)
==
(cond
  [(empty? (cons 4 '())) '()]
  [else
    (cond
      [(< (first (cons 4 '())) 5)
        (cons (first (cons 4 '()))
              (extract < (rest (cons 4 '())) 5))]
      [else (extract < (rest (cons 4 '())) 5)]]])

```

やはり、R は <, l は (cons 4 '()), t は 5 に置き換えられます。残りは簡単です：

```

(cond
  [(empty? (cons 4 '())) '()]
  [else
    (cond
      [(< (first (cons 4 '())) 5)
        (cons (first (cons 4 '()))
              (extract < (rest (cons 4 '())) 5))]
      [else (extract < (rest (cons 4 '())) 5)]]])
==
(cond
  [#false '()]
  [else
    (cond
      [(< (first (cons 4 '())) 5)
        (cons (first (cons 4 '()))
              (extract < (rest (cons 4 '())) 5))]
      [else (extract < (rest (cons 4 '())) 5)]]])
==
(cond
  [(< (first (cons 4 '())) 5)

```

```

      (cons (first (cons 4 '()))
            (extract < (rest (cons 4 '())) 5)))
[else (extract < (rest (cons 4 '())) 5)]]
==
(cond
  [(< 4 5)
   (cons (first (cons 4 '()))
         (extract < (rest (cons 4 '())) 5))]
  [else (extract < (rest (cons 4 '())) 5)]]

```

ここが重要なステップであり、代入された位置で < が実際に使用されています。そして、計算が続きます：

```

==
(cond
  [#true
   (cons (first (cons 4 '()))
         (extract < (rest (cons 4 '())) 5))]
  [else (extract < (rest (cons 4 '())) 5)]]
==
(cons 4 (extract < (rest (cons 4 '())) 5))
==
(cons 4 (extract < '() 5))
==
(cons 4 '())

```

最後の例は、2つの要素のリストへの適用です：

```

(extract < (cons 6 (cons 4 '())) 5)
== (extract < (cons 4 '()) 5)
== (cons 4 (extract < '() 5))
== (cons 4 '())

```

ステップ 1 は、閾値未満でない場合に extract がリストの最初の要素を除外するケースを扱っています。

Exercise 246. DrRacket の Stepper (ステップ実行) を使用して、最後の計算のステップ 1 を確認せよ。

Exercise 247. DrRacket の Stepper を使用して (extract < (cons 8 (cons 4 '())) 5) を評価せよ。

Exercise 248. DrRacket の Stepper で (squared>? 3 10) と (squared>? 4 10) を評価せよ。

次の対話を考えてみます：

```

> (extract squared>? (list 3 4 5) 10)
(list 4 5)

```

Stepper が示すステップの一部を以下に示します：

```

(extract squared>? (list 3 4 5) 10)
; (1) ==
(cond
  [(empty? (list 3 4 5)) '()]
  [else
   (cond
    [(squared>? (first (list 3 4 5)) 10)
     (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
    [else (extract squared>? (rest (list 3 4 5)) 10)]]])

; (2) == ... ==
(cond
  [(squared>? 3 10)
   (cons (first (list 3 4 5)) (extract squared>? (rest (list 3 4 5)) 10))]
  [else (extract squared>? (rest (list 3 4 5)) 10)])

```

Stepper を使用して、行 (1) から (2) へのステップを確認せよ。さらにステップを進めて、ステップ (2) から最終結果までの隙間を埋めよ。各ステップを代入モデルや関数の評価ルールを用いて説明せよ。

Exercise 249. 関数は「値」であり、引数、結果、リストの要素になり得ます。以下の定義と式を DrRacket の定義ウィンドウに入力し、Stepper を使ってこのプログラムがどのように動作するかを調べよ：

```

(define (f x) x)
(cons f '())
(f f)
(cons f (cons 10 (cons (f 10) '())))

```

15 抽象化の設計 (Designing Abstractions)

本質的に、抽象化とは「具体的なものをパラメータに変換すること」です。前章でもこれを行いました。類似した関数定義を抽象化するには、定義内の具体的な値を置き換えるパラメータを追加します。類似したデータ定義を抽象化するには、パラメータ化されたデータ定義を作成します。他のプログラミング言語に触れると、その抽象化メカニズムもパラメータの導入を必要とすることを目にするでしょう（それが関数の引数であるとは限りませんが）。

15.1 例からの抽象化 (Abstractions from Examples)

あなたが足し算を初めて学んだとき、具体的な例を使って作業しました。親はおそらく指を使って小さな2つの数を足すことを教えたでしょう。後になって、任意の2つの数を足す方法を学びました。そこで初めてある種の抽象化に触れました。さらに後で、摂氏から華氏への温度変換や、与えられた速度と時間で車が移動する距離を計算する式を定式化することを学びました。要するに、非常に具体的な例から抽象的な関係へと移行したのです。

```

; List-of-numbers -> List-of-numbers
; converts a list of Celsius temperatures to Fahrenheit
(define (cf* l)
  (cond
    [(empty? l) '()]
    [else
     (cons (C2F (first l)) (cf* (rest l))))])

; Inventory -> List-of-strings
; extracts the names of toys from an inventory
(define (names i)
  (cond
    [(empty? i) '()]
    [else
     (cons (IR-name (first i)) (names (rest i))))])

; Number -> Number
; converts one Celsius temperature to Fahrenheit
(define (C2F c)
  (+ (* 9/5 c) 32))

(define-struct IR [name price])
; An IR is a structure: (make-IR String Number)
; An Inventory is one of:
; - '()
; - (cons IR Inventory)

```

図 90: 似た 2 つの関数

本節では、例から抽象化を作成するための設計レシピを紹介します。抽象化を作成すること自体は簡単ですが、既存の抽象化を見つけて利用するという「難しい部分」は次の節に譲ります。

「類似性はどこにでもある」のエッセンスを思い出してください。2 つの具体的な定義から始め、それらを比較し、違いをマークし、そして抽象化します。抽象化を作成する手順は、ほとんどこれだけです： 1. 類似点と相違点の比較 (Compare) ほぼ同じで、名前と特定の位置にある具体的な値だけが異なる 2 つの関数定義を見つけたら、それらを比較して相違点をマークします。もし違いが複数箇所にある場合は、対応する違い同士を結びつけます。図 90 は、類似した関数定義のペアを示しています。2 つの関数はリストを処理し、各要素に関数を適用しています。違いは「どの関数を要素に適用するか」という点だけです。 2. パラメータ化 (Parameterize) 相違点としてマークした部分を新しいパラメータ名 (引数) に置き換え、パラメータリストに追加します。図 90 の例では、相違点を `g` という名前に置き換えます。これにより本質的な違いが排除されます。次に、非本質的な違い (関数名やリスト引数の名前など) を統一します。関数名を `map1`、リストパラメータを `k` と呼ぶことにすると、図 91 のように、完全に同一の関数定義が得られます。 racket

```

(define (map1 k g)
  (cond
    [(empty? k) '()]
    [else
     (cons
      (g (first k)) (map1 (rest k) g))]))

```

図 91: 抽象化された同一の関数定義

違いが複数ある場合は、対応する違いのラインごとに1つの追加パラメータを導入します。そして、すべての再帰呼び出し部分にも追加パラメータを忘れずに引き渡すように変更します。

3. 検証と再定義 (Validate) 新しい抽象関数が、元の関数の正しい抽象化であることを検証します。これは、元の関数を抽象関数を使って定義し直し、既存のテストスイートが正常に通るか確認することを意味します。元の関数が `f-original` で引数を1つ消費し、抽象関数が `abstract` であるとします。`f-original` が他方と `val` という値の使用において異なる場合、次の定義を行います：

```
(define (f-from-abstract x)
  (abstract x val))
```

この `f-from-abstract` が `f-original` と同等であることをテストします。先ほどの例に戻ると：

```
(define (cf*-from-map1 l) (map1 l C2F))
(define (names-from-map1 i) (map1 i IR-name))
```

元のテストケースが以下のように定義されていたとします：

```
(check-expect (cf* (list 100 0 -40)) (list 212 32 -40))
(check-expect (names (list (make-IR "doll" 21.0) (make-IR "bear" 13.0)))
  (list "doll" "bear"))
```

これらを次のように書き換えて実行し、成功を確認します：

```
(check-expect (cf*-from-map1 (list 100 0 -40)) (list 212 32 -40))
(check-expect (names-from-map1 (list (make-IR "doll" 21.0) (make-IR "bear"
13.0)))
  (list "doll" "bear"))
```

4. 署名の定式化 (Signature) 新しい抽象関数に署名 (Signature) を与えます。これについては 15.2 で詳しく扱います。

新しい抽象化を行ったら、他に使える用途がないか確認します。用途が多ければ、その抽象化は本当に有用です。たとえば、`map1` を使って数値リストの各要素に 1 を加える関数は以下のように定義できます：

```
; List-of-numbers -> List-of-numbers
(define (add1-to-each l)
  (map1 l add1))
```

このような汎用的な関数がすでに言語の一部として提供されていることもよくあります (Chapter 16 を参照)。

```
; Number -> [List-of Number]
; tabulates sin between n and 0 (incl.) in a list
(define (tab-sin n)
  (cond
    [(= n 0) (list (sin 0))]
    [else (cons (sin n) (tab-sin (sub1 n)))]))
```

```

; Number -> [List-of Number]
; tabulates sqrt between n and 0 (incl.) in a list
(define (tab-sqrt n)
  (cond
    [(= n 0) (list (sqrt 0))]
    [else (cons (sqrt n) (tab-sqrt (sub1 n)))]))

```

図 92: 演習 250 のための類似した関数

```

; [List-of Number] -> Number
; computes the sum of the numbers on l
(define (sum l)
  (cond
    [(empty? l) 0]
    [else (+ (first l) (sum (rest l)))]))

; [List-of Number] -> Number
; computes the product of the numbers on l
(define (product l)
  (cond
    [(empty? l) 1]
    [else (* (first l) (product (rest l)))]))

```

図 93: 演習 251 のための類似した関数

Exercise 250. 図 92 の 2 つの関数の抽象化である `tabulate` を設計せよ。設計が完了したら、それを使って `sqr` (平方) と `tan` の表を作成する関数を定義せよ。

Exercise 251. 図 93 の 2 つの関数の抽象化である `fold1` を設計せよ。

Exercise 252. 図 94 の 2 つの関数の抽象化である `fold2` を設計せよ。この演習と演習 251 を比較せよ。どちらも `product` 関数を含んでいますが、この演習は 2 番目の関数 `image*` が `Posn` のリストを消費して `Image` を生成するため、難易度が高くなっています。得られた設計と前演習の設計を比較し、署名の抽象化についての洞察を得よ。

```

; [List-of Number] -> Number
(define (product l)
  (cond
    [(empty? l) 1]
    [else (* (first l) (product (rest l)))]))

; [List-of Posn] -> Image
(define (image* l)
  (cond
    [(empty? l) empty]
    [else (place-dot (first l) (image* (rest l)))]))

; Posn Image -> Image
(define (place-dot p img)
  (place-image dot (posn-x p) (posn-y p) img))

```

```
; 描画用定数
(define emt (empty-scene 100 100))
(define dot (circle 3 "solid" "red"))
```

図 94: 演習 252 のための類似した関数

15.2 署名における類似性 (Similarities in Signatures)

関数の署名 (Signature) は、その再利用の鍵となります。したがって、抽象化された関数を表現するのに十分な「最も一般的な署名」を記述する方法を学ぶ必要があります。

基本的に、署名はデータ定義と同じです。どちらもデータのクラスを指定しますが、データ定義はクラスに名前を付けるのに対し、署名はそうしません。それでも、以下のコード：

```
; Number Boolean -> String
(define (f n b) "hello world")
```

の最初の行は、Number と Boolean を消費して String を生成するすべての関数のクラスを表しています。

したがって、抽象化の設計レシピは署名にも適用できます。類似した署名を比較し、相違点を強調し、それらをパラメータで置き換えます。

cf* と names の元の署名を比較してみましょう： * cf* : [List-of Number] -> [List-of Number] * names : [List-of IR] -> [List-of String]

比較すると、矢印の左側で Number と IR が異なり、右側で Number と String が異なります。これら 2 つの相違点をパラメータ X と Y に置き換えると、次のようになります：

```
; [List-of X] -> [List-of Y]
```

このシグネチャに現れる変数 X と Y は、データ定義のパラメータ (Chapter 14.3) と同じようにデータのクラス全体を表します。これらをインスタンス化 (代入) すると、元の署名が得られます： * X と Y に Number を代入すると、cf* の署名になります。 * X に IR、Y に String を代入すると、names の署名になります。

次に、追加の関数引数を受け取る map1 の署名を考えます。map1 の署名は次のようになります：

```
; [X Y] [List-of X] [X -> Y] -> [List-of Y]
```

具体的には、map1 は X という型に属する要素のリストを消費します。また、X の要素を消費して Y という (また別の) 型の要素を生成する関数を消費します。そして、結果として Y のリストを返します。

もう 1 つの例として、演習 252 の product と image* の署名を抽象化します： * product の署名： [List-of Number] Number [Number Number -> Number] -> Number * image* の署名： [List-of Posn] Image [Posn Image -> Image] -> Image

これらと比較すると、すべての Number が一様に対応しているわけではなく： 1. ある Number は Posn に対応している（リストの要素）。 2. 別の Number は Image に対応している（ベースケースと結果の型）。

したがって、2つのパラメータ x と y を導入し、`fold2` の署名は次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
```

x と y の両方に Number を代入すると `product` の署名が得られ、 x に Posn、 y に Image を代入すると `image*` の署名が得られることを確認してください。

署名を抽象化するためのステップを以下にまとめます： 1. シグネチャの比較：2つの類似した関数の具体的な署名を比較し、相違点をマークします。 2. パラメータの抽出：抽象化によって追加されたパラメータが何を表しているかを特定し、対応するプレースホルダーを割り当てます。 3. 変数の置き換え：相違点を表す型パラメータ (x , y など) を導入し、統一された汎用的な署名を作成します。 4. 検証：型パラメータに具体的な型を代入し、元の個々の署名が正しく復元されることを確認します。

Exercise 253. 以下のそれぞれの署名は関数のクラスを表しています： 1. `[Number -> Boolean]` 2. `[Boolean String -> Boolean]` 3. `[Number Number Number -> Number]` 4. `[Number -> [List-of Number]]` 5. `[[List-of Number] -> Boolean]`

それぞれのクラスに属する具体的な関数の例を ISL で記述せよ。

Exercise 254. 以下の関数の署名を策定せよ： * `sort-n`: 数値のリストと、2つの数値を比較して真偽値を返す関数を消費し、ソートされた数値のリストを生成する。 * `sort-s`: 文字列のリストと、2つの文字列を比較して真偽値を返す関数を消費し、ソートされた文字列のリストを生成する。

その後、2つの署名を抽象化せよ。さらに、一般化された署名が、在庫レコード (IR) のリストをソートする関数の署名としてインスタンス化できることを示せ。

Exercise 255. 以下の関数の署名を策定せよ： * `map-n`: 数値のリストと、数値から数値への関数を消費し、数値のリストを生成する。 * `map-s`: 文字列のリストと、文字列から文字列への関数を消費し、文字列のリストを生成する。

その後、2つの署名を抽象化せよ。また、一般化された署名が上記 `map1` の署名としてインスタンス化できることを示せ。

15.3 単一の制御点 (Single Point of Control)

プログラムは文章の下書き (ドラフト) のようなものです。誤字の修正、文法エラーの修正、文章の一貫性の確保、そして重複の排除などの編集作業は、文章を良くするために不可欠です。誰も同じことを何度も繰り返す文章を読みたくはありませんし、そのようなプログラムも読みたくはありません。

重複を排除して抽象化を作成することには、多くの利点があります。定義がシンプルになり、既存の関数の潜在的な不具合（境界の不一致など）が明らかになります。しかし、最も重要な利点は、共通の機能に対して単一の制御点（Single Point of Control）を作ることです。

機能の定義を1箇所にとめることで、プログラムのメンテナンスが非常に容易になります：
* バグ（間違い）を発見したとき、1箇所だけを修正すればよくなります。
* 新しいデータ形式に対応する必要が生じたとき、1箇所にコードを追加するだけで済みます。
* アルゴリズムを改善したり高速化したりしたとき、1箇所の変更ですべての利用箇所が恩恵を受けます。

もしコードをコピー&ペーストしていた場合、すべてのコピー箇所を探し出して修正しなければならず、ミスの修正漏れが発生する危険性が非常に高くなります。

したがって、次のガイドラインを強く推奨します：

[!IMPORTANT] コードをコピーして修正する代わりに、抽象化を作成せよ。

抽象化の構築はプログラミングスキルを高めるための最も強力な練習の1つです。優れたプログラマは、重複を見つけると積極的にプログラムをリファクタリング（編集）し、新しい抽象化を構築して制御点を1箇所に集中させます。

15.4 テンプレートからの抽象化（Abstractions from Templates）

リストを処理する多くの関数は、データ定義から自動的に導かれる共通のテンプレートに従っています。したがって、関数が互いに似通った形になるのは必然です。

実は、私たちは個々の関数からだけでなく、テンプレートそのものから直接抽象化を作成することができます。リストを走査する関数の基本テンプレートを思い出してください：

```
(define (fun-for-l l)
  (cond
    [(empty? l) ...]
    [else (... (first l) ... (fun-for-l (rest l)) ...)]))
```

このテンプレートには、各節に1つずつ、合計2つの「空欄（gap）」があります。リスト処理関数を定義するとき、最初の節の空欄をベース値で埋め、2番目の節の空欄を「組み合わせる（結合する）関数」で埋めます。この結合関数は、リストの最初の要素と、残りのリストに対する再帰呼び出しの結果を組み合わせます。

この仕組みをパラメータ化して抽象関数を定義すると、次のようになります：

```
; [X Y] [List-of X] Y [X Y -> Y] -> Y
(define (reduce l base combine)
  (cond
    [(empty? l) base]
    [else
```

```
(combine (first l)
         (reduce (rest l) base combine)))]))
```

この `reduce` を使うと、リスト処理関数の多くを 1 行で定義できるようになります。たとえば、数値のリストの総和 `sum` と総乗 `product` は次のように再定義できます：

```
; [List-of Number] -> Number
(define (sum lon) (reduce lon 0 +))

; [List-of Number] -> Number
(define (product lon) (reduce lon 1 *))
```

`sum` では、空リストの場合のベース値は 0 であり、組み合わせる関数は `+` です。`product` では、ベース値は 1 であり、組み合わせる関数は `*` です。このように、多くのリスト処理関数が `reduce` を使って簡潔に再定義できます。

17 無名関数 (Nameless Functions)

17.1 lambda から生まれる関数 (Functions from lambda)

`lambda` の構文は次のとおりです：

```
(lambda (variable-1 ... variable-N) expression)
```

特徴はキーワード `lambda` です。続いて括弧で囲まれた変数列、そして結果を計算する式があります。

例 (1 引数)：

```
(lambda (x) (expt 10 x))
(lambda (x) (if (> x 0) x (- 0 x)))
(lambda (x) (string-append "hello " x))
```

これらは無名関数です。インタラクショナルエリアで直接使用できます。

17.2 lambda を使った計算 (Computing with lambda)

`lambda` 式は関数値として評価されます。

17.3 lambda を使った抽象化 (Abstracting with lambda)

`lambda` を使ってその場で関数を作成し、`map` や `filter` と組み合わせます。

```
(map (lambda (x) (* x x)) (list 1 2 3))
```

17.4 lambda を使った仕様 (Specifying with lambda)

テストケースや目的文で `lambda` を使って動作を指定できます。

17.5 lambda を使った表現 (Representing with lambda)

`lambda` は関数を値として表現するための強力な手段です。

Intermezzo 3: Scope and Abstraction

(今後の翻訳作業のためのプレースホルダー)

IV 絡み合ったデータ (Intertwined Data)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 4: The Nature of Numbers

(今後の翻訳作業のためのプレースホルダー)

V 生成的再帰 (Generative Recursion)

(今後の翻訳作業のためのプレースホルダー)

Intermezzo 5: The Cost of Computation

(今後の翻訳作業のためのプレースホルダー)

VI 蓄積子 (Accumulators)

(今後の翻訳作業のためのプレースホルダー)

エピローグ: 先へ進む (Epilogue: Moving On)

(今後の翻訳作業のためのプレースホルダー)